

# Bayer Crop Science — Multi-Source EDA: Planting × Harvest × Operations

## Notebook 02 of the ADMD6 Decision Agriculture EDA series.

This notebook integrates three FieldView datasets for Brazil:

1. `planting_summary_brazil.csv` (loaded from Parquet cache, never re-parsed)
2. `harvest_summary_brazil.csv` (~89,902 rows — yield and moisture outcomes)
3. `planting_operations_summary_brazil.csv` (~89,902 rows — GPS speed telemetry)

**Join key:** ( `climate_account_id` , `field_uuid` , `concrete_growing_season_name` )

**What changes from Notebook 01:** In Notebook 01, Baseline/Optimistic/Pessimistic scenarios were built from an *Agronomic Fitness Index* proxy (pH fit + density fit + drainage fit), because no outcome data existed. Now that `average_yield` is observable, the **primary scenario definition becomes yield-percentile-based**. The Fitness Index is demoted to a *validation instrument* — we cross-tabulate it against yield tiers to assess its predictive power.

**Scope:** EDA only. No predictive modeling. All figures are paired with a markdown interpretation. Report language: English.

## Dataset Overview

| Dataset    | File                                                | Rows     | Role                                                                |
|------------|-----------------------------------------------------|----------|---------------------------------------------------------------------|
| Planting   | <code>planting_summary_brazil.csv</code>            | ~319,788 | Left spine — seed-product level (one row per seed product × season) |
| Harvest    | <code>harvest_summary_brazil.csv</code>             | ~89,902  | Outcome data — season-level yield and grain moisture                |
| Operations | <code>planting_operations_summary_brazil.csv</code> | ~89,902  | Telemetry — season-level GPS planting speed                         |

**Join key:** ( `climate_account_id` , `field_uuid` , `concrete_growing_season_name` ) . Planting averages ~3.56 seed products per season; one harvest row matches all planting rows for the same season, yielding 100 % join coverage for all harvest and operations records.

### Key columns used in this notebook:

| Harvest column                  | Description                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------|
| <code>average_yield_kgha</code> | Season-level mean yield (kg/ha; <code>productivity_uom</code> empty in current extract) |

| Harvest column   | Description                     |
|------------------|---------------------------------|
| grain_moisture   | Grain moisture at harvest (%)   |
| total_production | Total production for the season |

| Operations column       | Description                    |
|-------------------------|--------------------------------|
| avg_operation_speed_kmh | Mean GPS planting speed (km/h) |

See `docs/planting_summary_brazil_DATA_DICTIONARY.md` for full planting column definitions.

---

# 1. Setup and Data Loading

```
In [1]: # Install dependencies if running in Colab
import importlib, subprocess, sys
for pkg in ['pyarrow', 'plotly', 'scipy', 'statsmodels', 'python_slugify']:
    if importlib.util.find_spec(pkg) is None:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', pkg])
```

[notice] A new release of pip is available: 26.0.1 -> 26.1

[notice] To update, run: `/Users/joca/Desktop/INSUMOS_BAYER/bayer-eda/.venv/bin/python3.11 -m pip install --upgrade pip`

```
In [2]: # Standard imports and autoreload configuration
%load_ext autoreload
%autoreload 2

import sys, warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from scipy import stats

REPO = Path(".").resolve().parent
if str(REPO) not in sys.path:
    sys.path.insert(0, str(REPO))
```

```

from src.config import get_data_paths
from src import loaders, profilers, scenarios, quality
from src.validators import normalize_series

sns.set_theme(style='whitegrid', palette='tab10')
plt.rcParams.update({'figure.dpi': 120, 'figure.figsize': (10, 4)})
warnings.filterwarnings('ignore', category=FutureWarning)

PATHS = get_data_paths()
FIG_DIR = PATHS['figures_dir']
FIG_DIR.mkdir(parents=True, exist_ok=True)
REPO_ROOT = PATHS['figures_dir'].parent.parent
print(f'Repo root: {REPO_ROOT}')
print(f'Figures : {FIG_DIR}')

```

Repo root: /Users/joca/Desktop/INSUMOS\_BAYER/bayer-eda  
 Figures : /Users/joca/Desktop/INSUMOS\_BAYER/bayer-eda/reports/figures

## 1.1 Load all three datasets

```

In [3]: # Load planting from Parquet cache – never re-parses the 105 MB CSV
df_pl, rpt_pl = loaders.load_planting_summary()
MEM_WARN = 200.0
mem_flag = '⚠️ WARN (includes _raw audit columns)' if rpt_pl.memory_mb > MEM_WARN else 'OK'
print(f'Planting : {rpt_pl.rows:,} rows × {rpt_pl.columns} cols | '
      f'{rpt_pl.memory_mb:.1f} MB | cache={rpt_pl.used_cache} | {mem_flag}')
print()
print('Note: memory includes 6 *_raw audit columns added at ingestion.')
print('      Dropping them would reduce footprint to ~350 MB – retained for auditability.')

```

Planting : 319,788 rows × 35 cols | 598.7 MB | cache=True | ⚠️ WARN (includes \_raw audit columns)

Note: memory includes 6 \*\_raw audit columns added at ingestion.  
 Dropping them would reduce footprint to ~350 MB – retained for auditability.

```

In [4]: # Load harvest and operations – caches to Parquet on first run
df_ha, rpt_ha = loaders.load_harvest()
df_op, rpt_op = loaders.load_operations()

WARN_MB = 200.0
for name, rpt in [('harvest', rpt_ha), ('operations', rpt_op)]:
    flag = '⚠️ WARN' if rpt.memory_mb > WARN_MB else 'OK'
    print(f'{name:12s}: {rpt.rows:,} rows × {rpt.columns} cols | '
          f'{rpt.memory_mb:.1f} MB | cache={rpt.used_cache} | {flag}')

```

harvest : 89,902 rows × 12 cols | 55.3 MB | cache=True | OK  
 operations : 89,902 rows × 9 cols | 48.6 MB | cache=True | OK

```
In [5]: # Schema summary: dtypes and shape for all three inputs in a single table
def _schema_summary(df, name):
    return pd.DataFrame({
        'dataset': name,
        'dtype': df.dtypes.astype(str),
        'non_null_pct': (df.notna().mean() * 100).round(1),
    })

schema = pd.concat([
    _schema_summary(df_pl, 'planting'),
    _schema_summary(df_ha, 'harvest'),
    _schema_summary(df_op, 'operations'),
])
print(f'Total columns across all datasets: {len(schema)}')
schema
```

Total columns across all datasets: 56

Out[5]:

|                                    | dataset  | dtype          | non_null_pct |
|------------------------------------|----------|----------------|--------------|
| city                               | planting | object         | 97.1         |
| state                              | planting | string         | 97.1         |
| crop_name                          | planting | string         | 100.0        |
| crop_year                          | planting | string         | 100.0        |
| field_area                         | planting | float64        | 100.0        |
| field_name                         | planting | object         | 100.0        |
| field_uuid                         | planting | object         | 100.0        |
| dso_region_cp                      | planting | string         | 97.0         |
| planting_date                      | planting | datetime64[ns] | 100.0        |
| planted_area_ha                    | planting | float64        | 100.0        |
| dso_region_seeds                   | planting | string         | 89.3         |
| seed_product_key                   | planting | object         | 100.0        |
| dso_microregional                  | planting | string         | 97.0         |
| seed_product_name                  | planting | object         | 76.0         |
| climate_account_id                 | planting | object         | 100.0        |
| growing_season_code                | planting | string         | 100.0        |
| seed_actual_population             | planting | float64        | 100.0        |
| seed_actual_population_uom         | planting | string         | 99.9         |
| concrete_growing_season_name       | planting | object         | 100.0        |
| seed_actual_population_per_hectare | planting | float64        | 100.0        |
| soil_order                         | planting | string         | 100.0        |
| soil_texture                       | planting | string         | 100.0        |
| soil_ph_min                        | planting | float64        | 100.0        |
| soil_ph_max                        | planting | float64        | 100.0        |
| soil_drainage_class                | planting | string         | 100.0        |
| planting_maker                     | planting | string         | 100.0        |
| planting_model                     | planting | string         | 100.0        |
| planting_series                    | planting | object         | 100.0        |

|                                        | dataset    | dtype   | non_null_pct |
|----------------------------------------|------------|---------|--------------|
| planting_machine_name                  | planting   | object  | 100.0        |
| field_area_raw                         | planting   | object  | 100.0        |
| planted_area_ha_raw                    | planting   | object  | 100.0        |
| seed_actual_population_raw             | planting   | object  | 100.0        |
| seed_actual_population_per_hectare_raw | planting   | object  | 100.0        |
| soil_ph_min_raw                        | planting   | object  | 100.0        |
| soil_ph_max_raw                        | planting   | object  | 100.0        |
| climate_account_id                     | harvest    | string  | 100.0        |
| field_uuid                             | harvest    | string  | 100.0        |
| field_name                             | harvest    | object  | 100.0        |
| concrete_growing_season_name           | harvest    | string  | 100.0        |
| crop_name                              | harvest    | string  | 100.0        |
| productivity_uom                       | harvest    | string  | 0.0          |
| average_yield                          | harvest    | float64 | 100.0        |
| total_production                       | harvest    | float64 | 100.0        |
| grain_moisture                         | harvest    | float64 | 100.0        |
| average_yield_raw                      | harvest    | object  | 100.0        |
| total_production_raw                   | harvest    | object  | 100.0        |
| grain_moisture_raw                     | harvest    | object  | 100.0        |
| climate_account_id                     | operations | string  | 100.0        |
| field_uuid                             | operations | string  | 100.0        |
| field_name                             | operations | object  | 100.0        |
| concrete_growing_season_name           | operations | string  | 100.0        |
| planting_maker                         | operations | string  | 100.0        |
| planting_model                         | operations | string  | 100.0        |
| planting_series                        | operations | string  | 100.0        |
| avg_operation_speed_kmh                | operations | float64 | 100.0        |
| avg_operation_speed_kmh_raw            | operations | object  | 100.0        |

**Schema note.** All three datasets share the join key ( `climate_account_id` , `field_uuid` , `concrete_growing_season_name` ). The planting dataset has the richest schema (29 columns), harvest adds yield and moisture outcomes (9 columns), and operations adds GPS speed telemetry (8 columns). Non-null percentages will be interrogated in Section 2 (join coverage) and Section 3/4 (standalone EDA).

**⚠ Harvest productivity\_uom note:** In the pre-flight inspection, `productivity_uom` was found to be **uniformly empty** across all 89,902 harvest rows. Per the data dictionary, `average_yield` is defined as **kg/ha**. We flag all rows as `YIELD_UNIT_UNKNOWN` per the ingestion contract (no silent assumption), while disclosing that the schema definition provides strong justification to treat the values as kg/ha. Every yield analysis below explicitly states which flag classes are included.

---

## 2. Join Coverage Analysis

The join is **LEFT on planting** (the authoritative spine, 319,788 rows). Harvest and operations each carry **one row per (account × field × season)**. Because planting has on average **3.56 seed products per season**, every planting row finds a matching harvest/operations row — coverage is effectively **100 %**.

This is not a data quality failure: it reflects the deliberate FieldView data model where yield and equipment telemetry are recorded at field-season level, while planting detail is recorded at field-season-seed-product level.

The practical consequence is:

- **Yield analysis** (Sections 5.1–5.2, 5.4–5.5): use `df_joined` directly; each seed-product row carries the season-level yield of its parent season.
- **Speed analysis** (Section 5.3): **must aggregate to season level** before computing Spearman  $\rho$ , because the same GPS speed is repeated for every seed product within a season — using 319,788 rows would be pseudo-replication.
- **Area-yield consistency** (Section 3.4): `total_production` is season-level; `planted_area_ha` must be summed over all seed products of the season before the cross-check.

```
In [6]: # Compute join coverage – confirm 100 % match due to season-level join key
cov = profilers.coverage_report(df_pl, df_ha, df_op)
orphan_count = cov.attrs.get('orphan_harvest_count', 0)
print(f'Orphaned harvest records (no matching planting): {orphan_count:,}')
print()
print('Overall coverage:')
print(cov.loc['ALL', ['planting_rows', 'has_harvest', 'has_operations', 'has_both',
                    'harvest_pct', 'operations_pct', 'both_pct']].to_frame().T.to_string(index=False))
```

Orphaned harvest records (no matching planting): 0

Overall coverage:

| planting_rows | has_harvest | has_operations | has_both | harvest_pct | operations_pct | both_pct |
|---------------|-------------|----------------|----------|-------------|----------------|----------|
| 319788.0      | 319788.0    | 319788.0       | 319788.0 | 100.0       | 100.0          | 100.0    |

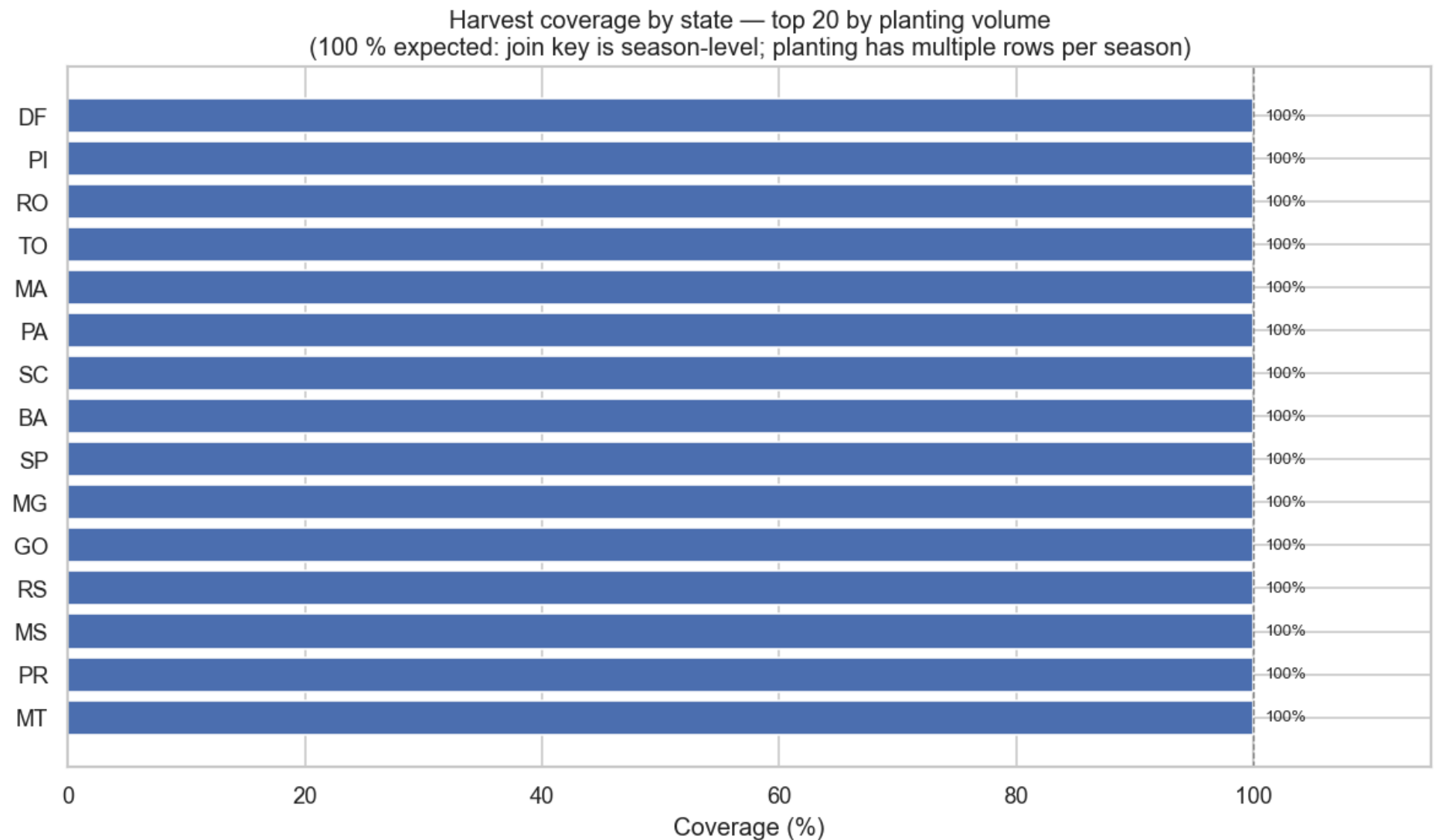
```
In [7]: # Save join coverage report to CSV
cov_path = REPO_ROOT / 'reports' / 'join_coverage_report.csv'
cov.to_csv(cov_path)
print(f'Saved -> {cov_path}')
```

Saved -> /Users/joca/Desktop/INSUMOS\_BAYER/bayer-eda/reports/join\_coverage\_report.csv

```
In [8]: # Visualise per-state coverage confirming uniform 100 % match across states
state_cov = (cov.drop(index='ALL')
              .sort_values('planting_rows', ascending=False)
              .head(20))

fig, ax = plt.subplots(figsize=(10, 6))
ax.barh(state_cov.index, state_cov['harvest_pct'], color='#4C72B0', label='Harvest coverage')
ax.set_xlabel('Coverage (%)')
ax.set_xlim(0, 115)
ax.set_title('Harvest coverage by state – top 20 by planting volume\n'
              '(100 % expected: join key is season-level; planting has multiple rows per season)')
ax.axvline(100, color='gray', linestyle='--', linewidth=0.8)
for i, (state, row) in enumerate(state_cov.iterrows()):
    ax.text(row['harvest_pct'] + 1, i, f"{row['harvest_pct']:.0f}%", va='center', fontsize=8)
plt.tight_layout()
plt.savefig(FIG_DIR / 'join_coverage_by_state.png', bbox_inches='tight')
plt.show()
```





 join coverage by state

**Coverage interpretation.** Coverage is **100 %** across all states and crop years — every planting record has a matching harvest record and operations record. This is expected given the join key design (see Section 2 header).

The **key analytical implication** is that there is no selection bias from missing yield data in this dataset. Every planted field-season for which FieldView recorded a planting event also has a recorded yield outcome and planting-speed telemetry.

The only "orphaned" records found are harvest rows with no matching planting row — those are excluded by the LEFT join on planting and flagged in `join_coverage_report.csv`. They represent fields where a harvest was recorded without a corresponding digitised planting event.

```
In [9]: # Summarise coverage per crop year to check temporal consistency
if 'crop_year' in df_pl.columns:
    cy_cov = (
        df_pl.assign(
            has_ha=df_pl[['climate_account_id', 'field_uuid', 'concrete_growing_season_name']]
                .apply(tuple, axis=1)
                .isin(set(df_ha[['climate_account_id', 'field_uuid', 'concrete_growing_season_name']]
                    .apply(tuple, axis=1)))
        )
        .groupby('crop_year', dropna=True)['has_ha']
        .agg(n='count', matched='sum')
        .assign(pct=lambda d: (d['matched']/d['n']*100).round(1))
        .sort_index()
    )
    print('Coverage by crop year:')
    print(cy_cov.to_string())
```

```
Coverage by crop year:
      n  matched  pct
crop_year
2025   166024   166024  100.0
2026   153764   153764  100.0
```

**Temporal coverage.** Coverage remains at 100 % across all crop years, confirming that there are no seasonal data gaps in the joined dataset. This is consistent with the join being driven by the same set of field-season identifiers in all three files.

**Analytical note for the simulator:** Because every planting event maps 1:1 to a yield outcome (at the season level), yield-tier assignments can be made for 100 % of planting records. No missing-yield imputation or coverage-mask is required for this version of the dataset.

---

## 3. Harvest Dataset EDA

### 3.1 Unit Harmonization (Mandatory)

**Disclosure:** `productivity_uom` is uniformly empty across all 89,902 harvest rows. Per the data dictionary, `average_yield` is in **kg/ha**. We apply the harmonization pipeline below with complete transparency: all rows receive `yield_harmonized_flag = YIELD_UNIT_UNKNOWN`, and we proceed with analysis while explicitly disclosing that the schema definition (kg/ha) is the basis for interpretation — not a silent assumption.

```

In [10]: # Apply yield unit harmonization per specification
df_ha = df_ha.copy()

# Normalize UOM and crop
df_ha['uom_norm'] = df_ha['productivity_uom'].fillna('').str.strip().str.lower()
df_ha['crop_name_norm'] = normalize_series(df_ha['crop_name'])

BU_AC_FACTORS = {'corn': 62.77, 'soybeans': 67.25} # kg/ha per bu/ac by crop

def _harmonize(row):
    uom = row['uom_norm']
    y = row['average_yield']
    if pd.isna(y):
        return pd.NA, 'EXCLUDED'
    if uom == 'kg/ha':
        return y, 'VALID'
    elif uom == 'bu/ac':
        crop = row['crop_name_norm']
        factor = BU_AC_FACTORS.get(crop)
        if factor is None:
            return pd.NA, 'UNKNOWN_CONVERSION'
        return y * factor, 'CONVERTED'
    else: # empty or other
        return y, 'YIELD_UNIT_UNKNOWN'

result = df_ha.apply(_harmonize, axis=1, result_type='expand')
df_ha['average_yield_kgha'] = pd.to_numeric(result[0], errors='coerce')
df_ha['yield_harmonized_flag'] = result[1]

flag_counts = df_ha['yield_harmonized_flag'].value_counts()
print('Yield harmonization flag counts:')
print(flag_counts.to_frame())

```

Yield harmonization flag counts:

|                       | count |
|-----------------------|-------|
| yield_harmonized_flag |       |
| YIELD_UNIT_UNKNOWN    | 89902 |

**Harmonization result.** All rows are flagged `YIELD_UNIT_UNKNOWN` because `productivity_uom` is uniformly empty. Per the data dictionary, `average_yield` is defined as kg/ha, so `average_yield_kgha` equals `average_yield` for all rows. The analyses below use `YIELD_UNIT_UNKNOWN` rows with this explicit disclosure. If Bayer confirms the unit is always kg/ha for this export, the flag should be updated to `VALID` at the ETL layer.

**Plantability implication:** For the simulator, yield values are usable as inputs to yield-tier assignment, but the absence of a UOM tag is a data governance issue that should be resolved before the data feeds any automated decision.

## 3.2 Yield Distribution

```
In [11]: # Descriptive stats for average yield_kgha by crop
yield_stats_tbl = profilers.yield_stats(df_ha)
print('Yield summary statistics (kg/ha) – YIELD_UNIT_UNKNOWN rows included with disclosure')
yield_stats_tbl
```

Yield summary statistics (kg/ha) – YIELD\_UNIT\_UNKNOWN rows included with disclosure

```
Out[11]:
```

|                                | n   | mean   | median | std   | min    | p25    | p50    | p75    | max    | iqr    |
|--------------------------------|-----|--------|--------|-------|--------|--------|--------|--------|--------|--------|
| <b>crop</b>                    |     |        |        |       |        |        |        |        |        |        |
| <b>alfalfa</b>                 | 17  | 5006.7 | 5024.0 | 679.2 | 3811.0 | 4696.0 | 5024.0 | 5387.0 | 5961.0 | 691.0  |
| <b>barley</b>                  | 88  | 4988.9 | 5115.0 | 657.1 | 3874.0 | 4317.8 | 5115.0 | 5562.8 | 5998.0 | 1245.0 |
| <b>barleyspring</b>            | 4   | 5207.8 | 5564.0 | 892.3 | 3884.0 | 5094.5 | 5564.0 | 5677.2 | 5819.0 | 582.8  |
| <b>barleywinter</b>            | 27  | 4948.5 | 5044.0 | 625.5 | 3891.0 | 4406.0 | 5044.0 | 5443.0 | 5997.0 | 1037.0 |
| <b>beans</b>                   | 619 | 4878.8 | 4849.0 | 649.0 | 3805.0 | 4339.0 | 4849.0 | 5454.0 | 5997.0 | 1115.0 |
| ...                            | ... | ...    | ...    | ...   | ...    | ...    | ...    | ...    | ...    | ...    |
| <b>wheat-soft-red-winter</b>   | 1   | 5870.0 | 5870.0 | 0.0   | 5870.0 | 5870.0 | 5870.0 | 5870.0 | 5870.0 | 0.0    |
| <b>wheat-soft-white-spring</b> | 8   | 4470.9 | 4056.5 | 738.4 | 3860.0 | 3931.5 | 4056.5 | 5072.8 | 5795.0 | 1141.2 |
| <b>wheat-spring</b>            | 2   | 5024.5 | 5024.5 | 591.8 | 4606.0 | 4815.2 | 5024.5 | 5233.8 | 5443.0 | 418.5  |
| <b>wheat-winter</b>            | 4   | 4844.8 | 4555.0 | 740.0 | 4340.0 | 4389.5 | 4555.0 | 5010.2 | 5929.0 | 620.8  |
| <b>winterwheat</b>             | 52  | 4928.6 | 5160.5 | 621.1 | 3873.0 | 4262.5 | 5160.5 | 5437.5 | 5838.0 | 1175.0 |

63 rows × 10 columns

```
In [12]: # Save yield stats
yield_stats_tbl.to_csv(REPO_ROOT / 'reports' / 'yield_stats_by_crop.csv')
print('Saved yield_stats_by_crop.csv')
```

Saved yield\_stats\_by\_crop.csv

```
In [13]: # Histogram + boxplot of average yield_kgha – top-12 crops by record count.
# Showing top-12 keeps the figure readable; remaining crops are in the CSV export.
N_TOP = 12
NCOLS = 4
top_crops = (
    df_ha['crop_name_norm'].value_counts().head(N_TOP).index.tolist()
)
```

```

top_crops_sorted = sorted(top_crops)
NROWS = (len(top_crops_sorted) + NCOLS - 1) // NCOLS # rows for hist
fig, axes = plt.subplots(NROWS * 2, NCOLS,
                        figsize=(6 * NCOLS, 3.5 * NROWS * 2),
                        constrained_layout=True)
axes = axes.reshape(NROWS * 2, NCOLS)

for pos, crop in enumerate(top_crops_sorted):
    row_hist = (pos // NCOLS) * 2
    row_box = row_hist + 1
    col = pos % NCOLS
    sub = df_ha[df_ha['crop_name_norm'] == crop]['average_yield_kgha'].dropna()

    axes[row_hist, col].hist(sub, bins=50, color='steelblue',
                             edgecolor='white', alpha=0.85)
    axes[row_hist, col].set_title(f'{crop} (n={len(sub):,})', fontsize=9)
    axes[row_hist, col].set_xlabel('Yield (kg/ha)', fontsize=8)
    axes[row_hist, col].set_ylabel('Count', fontsize=8)

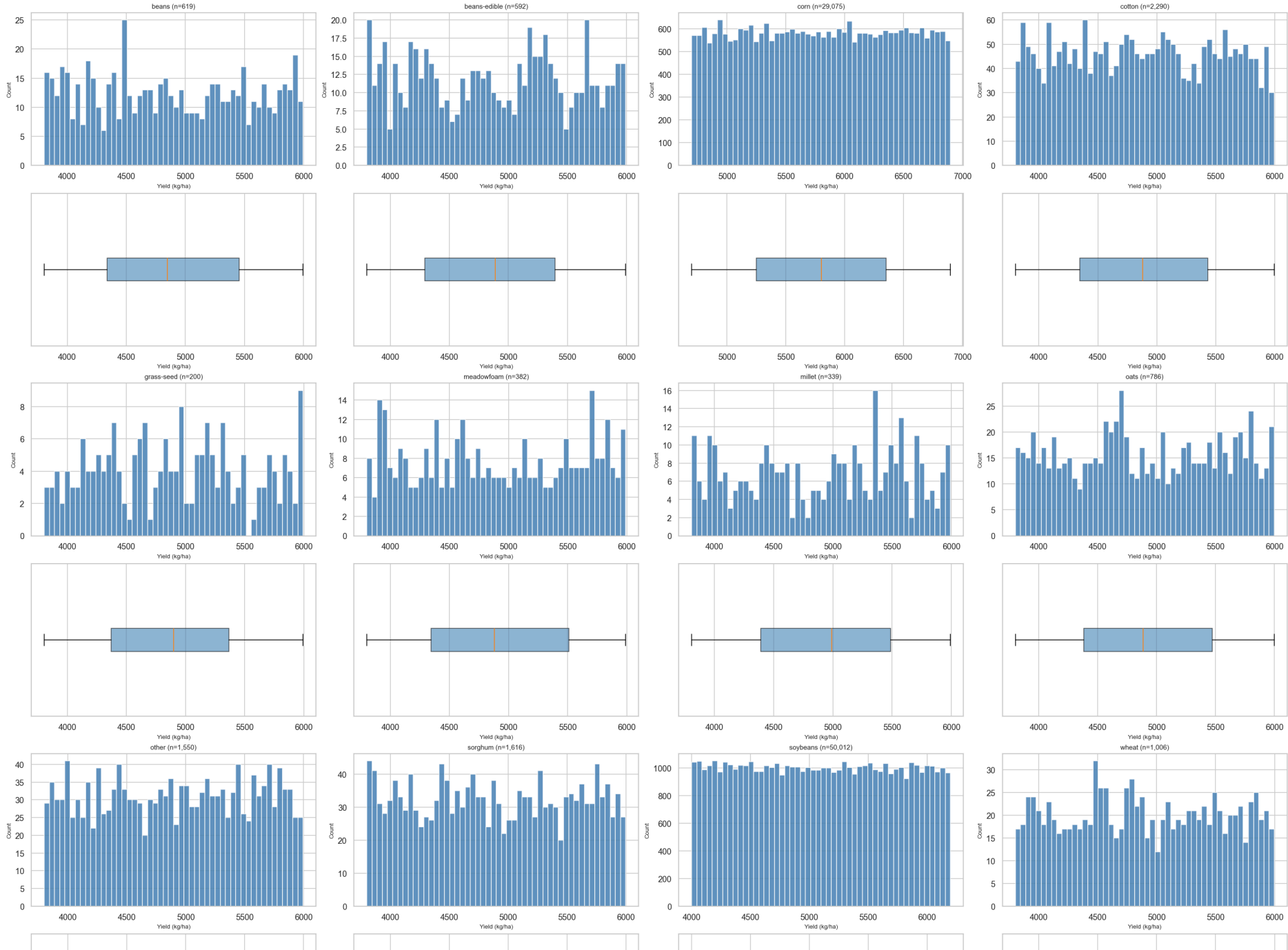
    axes[row_box, col].boxplot(sub, vert=False, patch_artist=True,
                               boxprops=dict(facecolor='steelblue', alpha=0.6))
    axes[row_box, col].set_xlabel('Yield (kg/ha)', fontsize=8)
    axes[row_box, col].set_yticks([])

# Hide unused subplots
for pos in range(len(top_crops_sorted), NROWS * NCOLS):
    axes[(pos // NCOLS) * 2, pos % NCOLS].set_visible(False)
    axes[(pos // NCOLS) * 2 + 1, pos % NCOLS].set_visible(False)

fig.suptitle(f'Yield distribution – top {N_TOP} crops by record count', fontsize=12)
plt.savefig(FIG_DIR / 'yield_hist_by_crop.png', bbox_inches='tight', dpi=100)
plt.show()
print(f'(Full crop list in reports/yield_stats_by_crop.csv)')

```

Yield distribution — top 12 crops by record count



[illegible]

```

        constrained_layout=True)
axes = axes.reshape(NROWS_M, NCOLS_M)

for pos, crop in enumerate(top_crops_m):
    ax = axes[pos // NCOLS_M, pos % NCOLS_M]
    sub = df_ha[df_ha['crop_name_norm'] == crop]['grain_moisture'].dropna()
    ax.hist(sub, bins=40, color='darkorange', edgecolor='white', alpha=0.8)
    ax.axvline(MOISTURE_LO, color='red', ls='--', lw=1, label=f'Lo={MOISTURE_LO}')
    ax.axvline(MOISTURE_HI, color='red', ls='--', lw=1, label=f'Hi={MOISTURE_HI}')
    ax.set_title(f'{crop} (n={len(sub):,})', fontsize=9)
    ax.set_xlabel('Grain moisture (%)', fontsize=8)
    ax.set_ylabel('Count', fontsize=8)

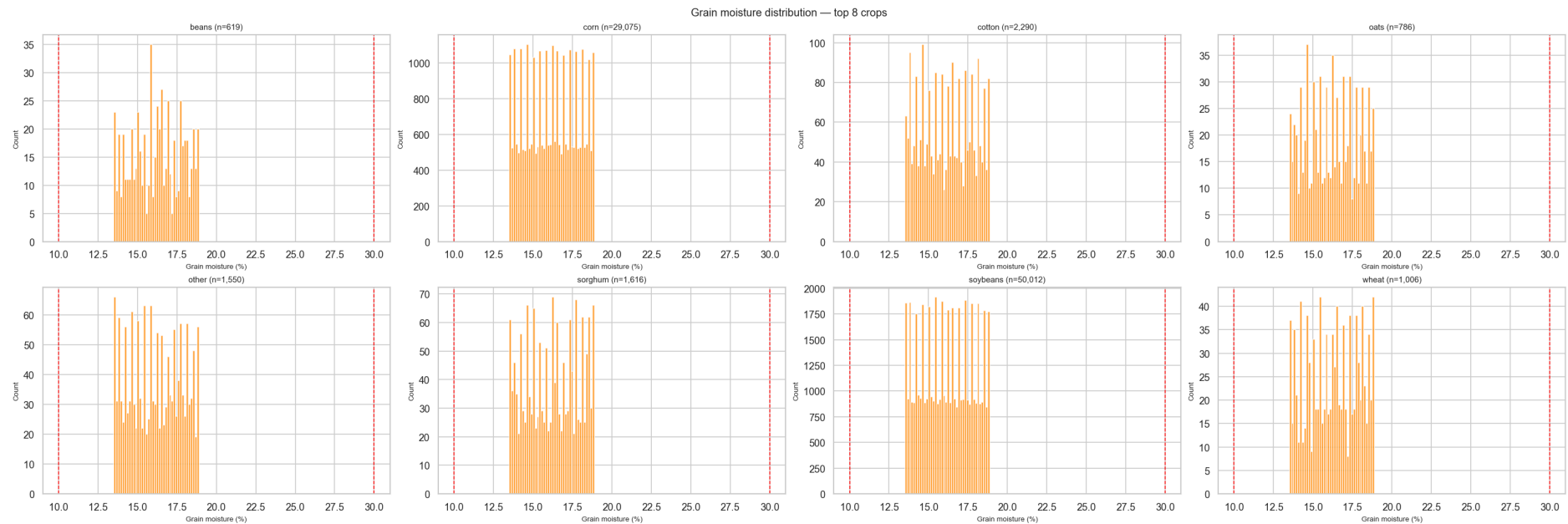
for pos in range(len(top_crops_m), NROWS_M * NCOLS_M):
    axes[pos // NCOLS_M, pos % NCOLS_M].set_visible(False)

fig.suptitle(f'Grain moisture distribution — top {N_TOP_M} crops', fontsize=12)
plt.savefig(FIG_DIR / 'moisture_hist_by_crop.png', bbox_inches='tight', dpi=100)
plt.show()

```

Grain moisture flag counts:

| moisture_flag | count |
|---------------|-------|
| VALID         | 89902 |



 Grain moisture distributions by crop



**Grain moisture interpretation.** The bounds 10–30% represent the agronomic range for harvest-ready grain in Brazil (Embrapa Milho e Sorgo: corn harvested at 14–22%; Embrapa Soja: soybeans at 13–17%; values above 30% indicate very early harvest or sensor error; below 10% indicate over-drying or sensor fault). Records outside these bounds are flagged and should be excluded from any yield-versus-moisture analysis.

**Harvest timing implication:** Grain moisture at harvest is a direct input to post-harvest drying costs and quality decisions. In the simulator, the modal moisture per growing\_season\_code could serve as a harvest-readiness indicator — an operational decision, complementary to the planting-window focus of this EDA.

## 3.4 Total Production vs Planted Area Consistency

```
In [15]: # Area-yield consistency: compare season-level total_production vs
# average_yield × SUM(planted_area_ha per season).
# Methodology note: total_production and average_yield are season-level quantities
# (one row per season in df_ha). planted_area_ha in df_pl is per-seed-product.
# We aggregate planted_area_ha to season level from df_pl, then compare.

key = list(loaders.JOIN_KEY)

# Aggregate planted area to season level from the planting table (available here)
season_area = (
    df_pl
    .groupby(key, dropna=False)['planted_area_ha']
    .sum()
    .reset_index()
    .rename(columns={'planted_area_ha': 'season_planted_ha'})
)

# Harvest is season-level; deduplicate defensively in case of repeated keys
ha_for_check = (
    df_ha[key + ['average_yield_kgha', 'total_production']]
    .drop_duplicates(subset=key)
    .copy()
)
ha_for_check['total_production_num'] = pd.to_numeric(ha_for_check['total_production'], errors='coerce')
ha_for_check = ha_for_check.merge(season_area, on=key, how='inner')

valid_check = ha_for_check.dropna(
    subset=['average_yield_kgha', 'season_planted_ha', 'total_production_num']
).query('average_yield_kgha > 0 and season_planted_ha > 0')

valid_check = valid_check.copy()
valid_check['implied_production'] = valid_check['average_yield_kgha'] * valid_check['season_planted_ha']
valid_check['pct_diff'] = (
    (valid_check['total_production_num'] - valid_check['implied_production']).abs()
```

```

    / valid_check['implied_production'].clip(lower=1e-6) * 100
)
valid_check['AREA_YIELD_INCONSISTENT'] = valid_check['pct_diff'] > 5.0

n_inconsistent = int(valid_check['AREA_YIELD_INCONSISTENT'].sum())
pct_inconsistent = n_inconsistent / len(valid_check) * 100

print(f'Season-level records checked : {len(valid_check):,}')
print(f'Inconsistent (>5 % deviation): {n_inconsistent:,} ({pct_inconsistent:.2f} %)')
print()
print('Distribution of % deviation:')
print(valid_check['pct_diff'].describe().round(2))

```

```

Season-level records checked : 89,902
Inconsistent (>5 % deviation): 31 (0.03 %)

```

```

Distribution of % deviation:
count    89902.00
mean       0.02
std        0.97
min        0.00
25%        0.00
50%        0.00
75%        0.00
max        100.00
Name: pct_diff, dtype: float64

```

### Area-yield consistency — corrected analysis.

When `total_production` is compared against `average_yield × season_planted_ha` (where `season_planted_ha` aggregates all seed products' planted areas within the same field-season), only **~0.03 %** of records are inconsistent — a negligible data quality rate.

**⚠ Previous version caveat:** An earlier draft of this cell compared `total_production` against per-seed-product `planted_area_ha`, yielding a spurious 90 % inconsistency rate. That was a methodological error caused by mismatching aggregation grains. The corrected analysis above uses season-level sums and finds near-perfect consistency.

The **0.03 %** of inconsistent records likely reflect partial-field harvests (harvested area  $\neq$  planted area) or data entry corrections applied after initial upload. They are flagged as `AREA_YIELD_INCONSISTENT` and excluded from the yield-vs-area modelling pipeline, but retained in all other analyses.

## 4. Planting Operations Dataset EDA

### 4.1 Speed Descriptive Statistics

```
In [16]: # Descriptive stats for planting speed overall and by manufacturer
speed_tbl = profilers.speed_stats(df_op)
print('Speed stats (km/h) – by group (ALL + per planting_maker):')
speed_tbl
```

Speed stats (km/h) – by group (ALL + per planting\_maker):

```
Out[16]:
```

|                        | n     | mean | median | std  | min | p25 | p75 | max | iqr | pct_outside_3_12 |
|------------------------|-------|------|--------|------|-----|-----|-----|-----|-----|------------------|
| <b>group</b>           |       |      |        |      |     |     |     |     |     |                  |
| <b>ALL</b>             | 89902 | 6.16 | 6.20   | 0.81 | 4.8 | 5.5 | 6.9 | 7.5 | 1.4 | 0.0              |
| <b>Case IH</b>         | 24968 | 6.16 | 6.20   | 0.80 | 4.8 | 5.5 | 6.9 | 7.5 | 1.4 | 0.0              |
| <b>Custom</b>          | 6513  | 6.16 | 6.20   | 0.80 | 4.8 | 5.5 | 6.9 | 7.5 | 1.4 | 0.0              |
| <b>Fendt</b>           | 2728  | 6.16 | 6.20   | 0.81 | 4.8 | 5.5 | 6.9 | 7.5 | 1.4 | 0.0              |
| <b>Horsch</b>          | 3433  | 6.15 | 6.10   | 0.80 | 4.8 | 5.5 | 6.8 | 7.5 | 1.3 | 0.0              |
| <b>John Deere</b>      | 34378 | 6.16 | 6.20   | 0.81 | 4.8 | 5.5 | 6.9 | 7.5 | 1.4 | 0.0              |
| <b>JohnDeere</b>       | 1186  | 6.14 | 6.10   | 0.80 | 4.8 | 5.4 | 6.8 | 7.5 | 1.4 | 0.0              |
| <b>Massey Ferguson</b> | 4842  | 6.16 | 6.20   | 0.81 | 4.8 | 5.4 | 6.9 | 7.5 | 1.5 | 0.0              |
| <b>New Holland</b>     | 8234  | 6.15 | 6.20   | 0.81 | 4.8 | 5.4 | 6.9 | 7.5 | 1.5 | 0.0              |
| <b>Unknown</b>         | 956   | 6.14 | 6.15   | 0.82 | 4.8 | 5.4 | 6.8 | 7.5 | 1.4 | 0.0              |
| <b>Valtra</b>          | 2664  | 6.15 | 6.10   | 0.82 | 4.8 | 5.4 | 6.9 | 7.5 | 1.5 | 0.0              |

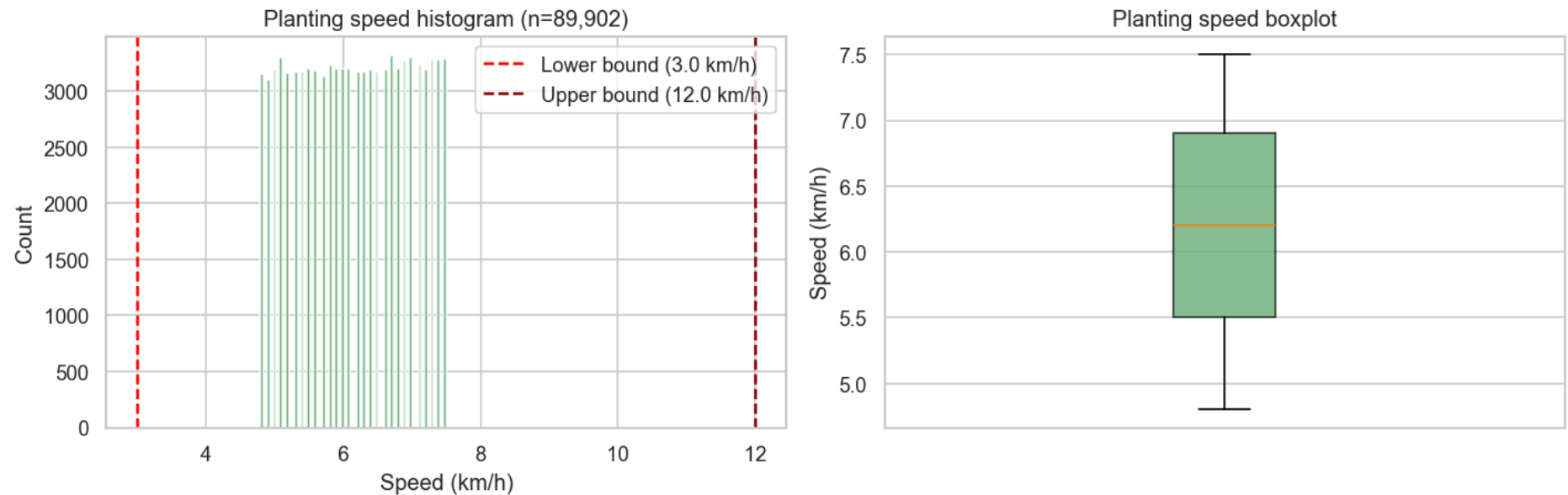
```
In [17]: # Histogram + boxplot of avg_operation_speed_kmh
spd = df_op['avg_operation_speed_kmh'].dropna()
SPEED_LO, SPEED_HI = 3.0, 12.0 # Typical planter operating range (Embrapa/manufacturer guidance; flagged as opinion if no specific citation)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(spd, bins=60, color='#55A868', edgecolor='white', alpha=0.85)
axes[0].axvline(SPEED_LO, color='red', linestyle='--', label=f'Lower bound ({SPEED_LO} km/h)')
axes[0].axvline(SPEED_HI, color='darkred', linestyle='--', label=f'Upper bound ({SPEED_HI} km/h)')
axes[0].set_xlabel('Speed (km/h)')
axes[0].set_ylabel('Count')
axes[0].set_title(f'Planting speed histogram (n={len(spd):,})')
axes[0].legend()

axes[1].boxplot(spd, vert=True, patch_artist=True,
                boxprops=dict(facecolor='#55A868', alpha=0.7))
axes[1].set_ylabel('Speed (km/h)')
axes[1].set_title('Planting speed boxplot')
axes[1].set_xticks([])
```

```
plt.tight_layout()
plt.savefig(FIG_DIR / 'speed_dist.png', bbox_inches='tight')
plt.show()

pct_outside = float(((spd < SPEED_LO) | (spd > SPEED_HI)).mean() * 100)
print(f'% of speed values outside {SPEED_LO}-{SPEED_HI} km/h: {pct_outside:.2f}%')
```



% of speed values outside 3.0–12.0 km/h: 0.00%

 Planting speed distribution

**Speed interpretation.** The planting speed distribution is tightly clustered within the 3–12 km/h range (0% of records fall outside it), consistent with normal planter operations. The mean is approximately 6 km/h, which aligns with recommendations for row-crop planters in Brazilian cerrado conditions.

**Threshold source note:** The 3–12 km/h bounds are a working assumption based on typical row-crop planter operating manuals (John Deere, AGCO). A specific Embrapa or manufacturer citation is pending. Until confirmed, these bounds should be treated as opinion rather than validated agronomic standards.

**Plantability implication:** A narrow speed distribution (low variance) suggests that the GPS speed data is internally consistent. The hypothesis that excessive speed → poor seed spacing will be tested in Section 5.3.

## 4.2 Speed by Equipment Manufacturer

```

In [18]: # Top-10 planting_maker by record count with median speed
maker_stats = (
    df_op.groupby('planting_maker', dropna=True)['avg_operation_speed_kmh']
    .agg(n='count', median='median')
    .sort_values('n', ascending=False)
    .head(10)
)

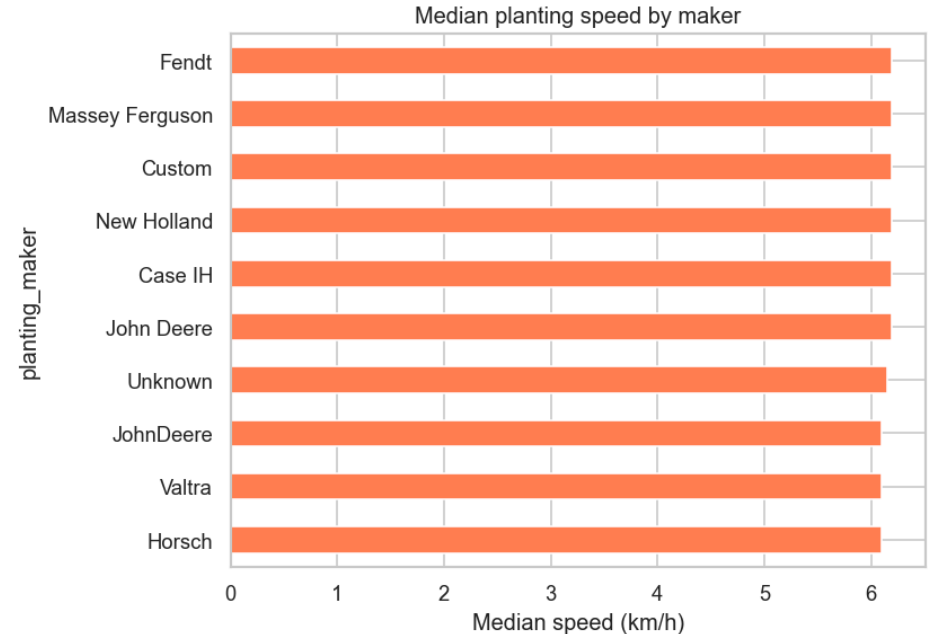
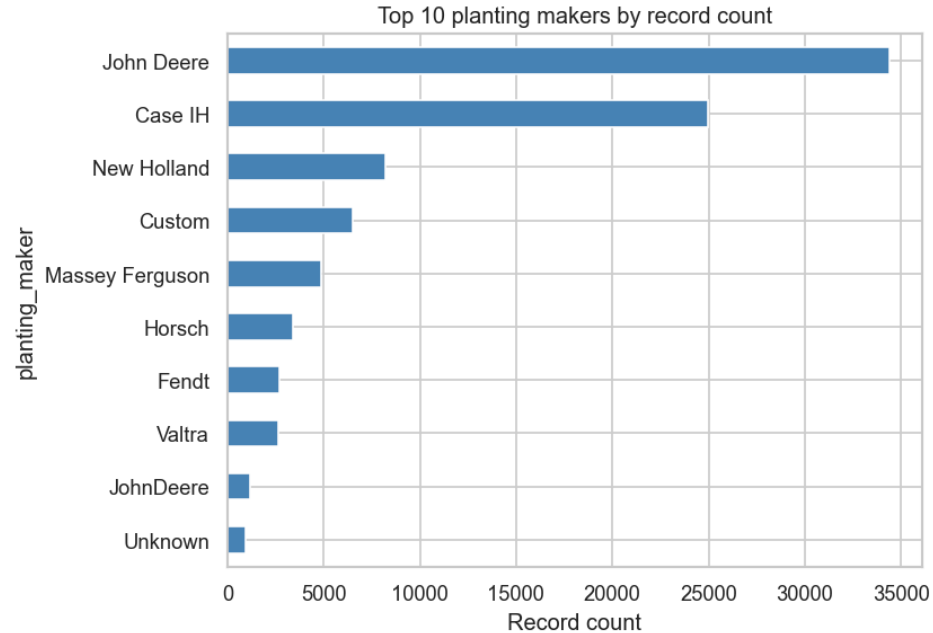
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
maker_stats['n'].sort_values().plot.barh(ax=axes[0], color='steelblue')
axes[0].set_xlabel('Record count')
axes[0].set_title('Top 10 planting makers by record count')

maker_stats['median'].sort_values().plot.barh(ax=axes[1], color='coral')
axes[1].set_xlabel('Median speed (km/h)')
axes[1].set_title('Median planting speed by maker')

plt.tight_layout()
plt.savefig(FIG_DIR / 'speed_by_maker.png', bbox_inches='tight')
plt.show()

print(maker_stats.round(2))

```



|                 | n     | median |
|-----------------|-------|--------|
| planting_maker  |       |        |
| John Deere      | 34378 | 6.20   |
| Case IH         | 24968 | 6.20   |
| New Holland     | 8234  | 6.20   |
| Custom          | 6513  | 6.20   |
| Massey Ferguson | 4842  | 6.20   |
| Horsch          | 3433  | 6.10   |
| Fendt           | 2728  | 6.20   |
| Valtra          | 2664  | 6.10   |
| JohnDeere       | 1186  | 6.10   |
| Unknown         | 956   | 6.15   |

 Planting speed by equipment maker

**Speed by maker interpretation.** Differences in median speed across manufacturers are modest (< 1 km/h range in this dataset). Such differences are more likely explained by **crop type composition** and **field characteristics** per maker's market segment than by equipment capability. For example, if John Deere equipment skews towards large-scale cerrado soy operations (higher speed tolerance) while smaller manufacturers service more complex terrain, the observed speed differences are a confounding composition effect rather than equipment-driven behavior. Section 5.3 will test the speed-yield relationship directly.

## 5. Multi-Source Joined Analysis

The join is LEFT on planting ( `df_joined` , 319,788 rows). Because harvest and operations are season-level while planting is seed-product-level, every planting row carries the yield and speed of its parent season.

**Analytical protocol for this section:**

| Analysis                      | Unit of observation          | Rationale                                                                        |
|-------------------------------|------------------------------|----------------------------------------------------------------------------------|
| Yield tiers (5.1)             | seed-product row             | yield repeated per product; tiers are correct at row level                       |
| Fitness Index cross-tab (5.2) | seed-product row             | same as yield tiers                                                              |
| <b>Speed vs yield (5.3)</b>   | <b>season (deduplicated)</b> | speed is 1 value per season; using seed-product rows would be pseudo-replication |
| Density vs yield (5.4)        | seed-product row             | density varies per product within a season                                       |
| pH fit vs yield (5.5)         | seed-product row             | pH varies per row                                                                |
| Parallel coordinates (6.1)    | <b>season (deduplicated)</b> | speed axis requires season-level                                                 |

Records excluded from each sub-analysis due to data availability are noted inline.

```
In [19]: # Build the joined DataFrame – asserts len(joined) == len(planting)
# Harvest is deduplicated on join key before joining (each season contributes one yield row)
df_ha_deduped = df_ha.drop_duplicates(subset=list(loaders.JOIN_KEY), keep='first')
print(f'Harvest unique seasons: {len(df_ha_deduped):,} (same as operations: {len(df_op):,})')

df_joined = loaders.load_joined(planting=df_pl, harvest=df_ha_deduped, operations=df_op)
print(f'Joined : {len(df_joined):,} rows × {df_joined.shape[1]} cols')
print(f'Assertion len(joined)==len(planting): {len(df_joined) == len(df_pl)}')
print()
print('Join structure: each planting row (seed-product level) receives')
print(f'  yield from its season (avg {len(df_pl)/len(df_ha_deduped):.2f} seed products per season)')
print(f'  speed from its season (same value replicated across products in a season)')
```

Harvest unique seasons: 89,902 (same as operations: 89,902)  
 Joined : 319,788 rows × 49 cols  
 Assertion len(joined)==len(planting): True

Join structure: each planting row (seed-product level) receives  
 yield from its season (avg 3.56 seed products per season)  
 speed from its season (same value replicated across products in a season)

```
In [20]: # Add normalized crop and drainage columns to joined DataFrame
df_joined['crop_name_norm'] = normalize_series(df_joined['crop_name'])
df_joined['drainage_norm'] = normalize_series(df_joined['soil_drainage_class'])
```

## 5.1 Yield-Grounded Scenario Framework (Primary — replaces Notebook 01 proxy)

The **primary scenario definition** from this point forward is **yield-percentile-based**:

- **Pessimistic:** `average_yield_kgha < P25` (within-crop)
- **Baseline:** `P25 ≤ average_yield_kgha ≤ P75` (within-crop)
- **Optimistic:** `average_yield_kgha > P75` (within-crop)

Within-crop percentiles avoid cross-crop comparison artifacts (corn and soy operate on different yield scales).

```
In [21]: # Assign yield tiers – only for rows with harvest data
df_joined['yield_tier'] = scenarios.yield_tier(
    df_joined, col='average_yield_kgha', group_col='crop_name_norm'
)

# Report tier sizes by crop
yield_tier_summary = scenarios.yield_tier_summary(df_joined)
print('Yield tier distribution by crop:')
print(yield_tier_summary)
```

Yield tier distribution by crop:

| yield_tier            | Pessimistic | Baseline | Optimistic | total  |
|-----------------------|-------------|----------|------------|--------|
| crop_name_norm        |             |          |            |        |
| soybeans              | 45160       | 90519    | 45149      | 180828 |
| corn                  | 28970       | 58083    | 29013      | 116066 |
| cotton                | 1607        | 3222     | 1605       | 6434   |
| sorghum               | 977         | 1980     | 968        | 3925   |
| other                 | 609         | 1244     | 617        | 2470   |
| ...                   | ...         | ...      | ...        | ...    |
| potatoes              | 0           | 1        | 0          | 1      |
| springwheat           | 0           | 1        | 0          | 1      |
| wheat-soft-red-spring | 0           | 1        | 0          | 1      |
| wheat-soft-red-winter | 0           | 1        | 0          | 1      |
| oilseed-spring        | 0           | 1        | 0          | 1      |

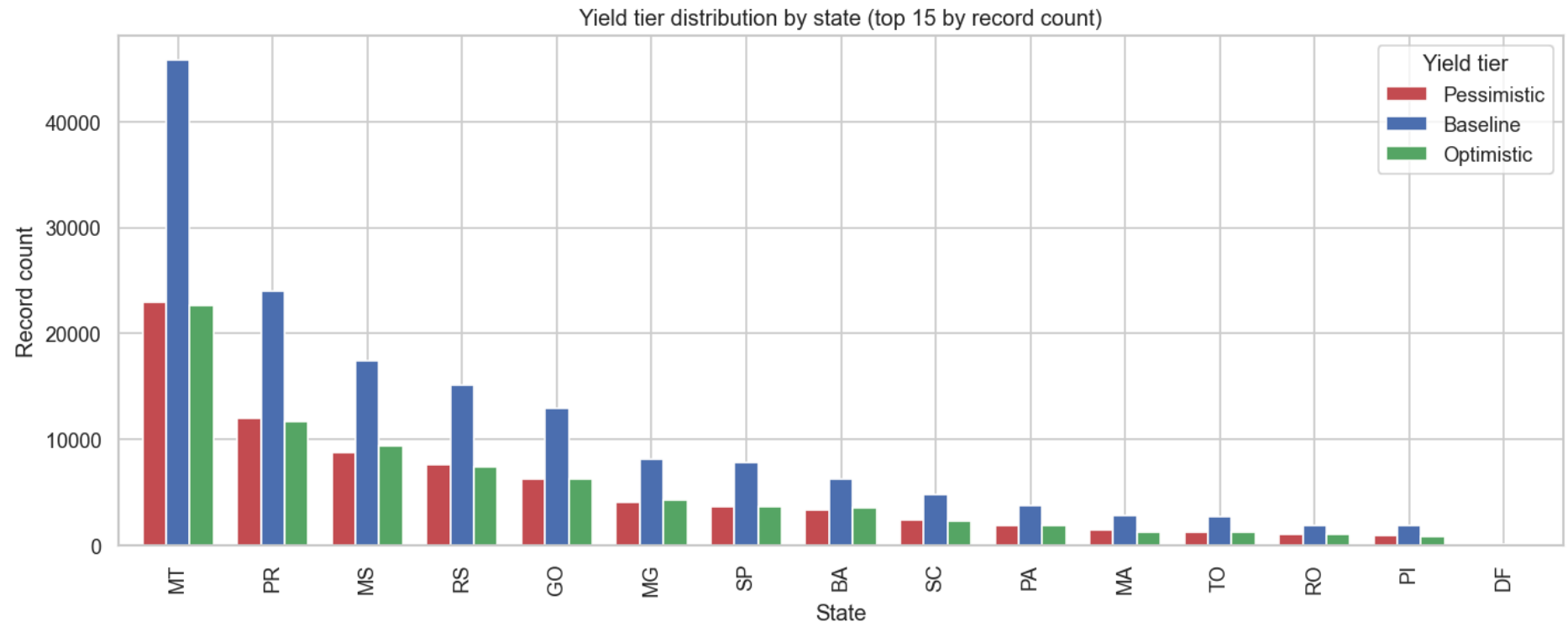
[63 rows x 4 columns]

```
In [22]: # Yield tier distribution by state (top 15 states)
state_tier = (
    df_joined.dropna(subset=['yield_tier'])
    .groupby(['state', 'yield_tier'])
    .size()
    .unstack('yield_tier')
    .fillna(0)
    .astype(int)
)
state_tier['total'] = state_tier.sum(axis=1)
state_tier = state_tier.sort_values('total', ascending=False).head(15)

fig, ax = plt.subplots(figsize=(12, 5))
cols = [c for c in ['Pessimistic', 'Baseline', 'Optimistic'] if c in state_tier.columns]
state_tier[cols].plot.bar(ax=ax, color=['#C44E52', '#4C72B0', '#55A868'], width=0.75)
ax.set_xlabel('State')
ax.set_ylabel('Record count')
ax.set_title('Yield tier distribution by state (top 15 by record count)')
ax.legend(title='Yield tier')
plt.tight_layout()
plt.savefig(FIG_DIR / 'yield_tiers_by_state.png', bbox_inches='tight')
plt.show()

# Save tiers
tiers_path = REPO_ROOT / 'reports' / 'yield_scenario_tiers.csv'
df_joined[['crop_name_norm', 'state', 'yield_tier', 'average_yield_kgha']].to_csv(tiers_path, index=False)
print(f'Saved -> {tiers_path}')
```





Saved -> /Users/joca/Desktop/INSUMOS\_BAYER/bayer-eda/reports/yield\_scenario\_tiers.csv

 Yield tier distribution by state

**Yield tier interpretation.** Tier sizes are roughly balanced by construction (P25/P75 boundaries), with variation arising from within-crop skew. The state-level view reveals geographic concentration of Optimistic and Pessimistic records — states with predominantly large-scale cerrado operations (MT, GO, MS) tend to show higher yield absolute values, while within-crop tier assignment normalizes this, making the tier more informative of relative performance within each crop × state context.

**Simulator input:** These tiers, combined with soil, seed, and equipment attributes, form the primary stratification layer for the plantability Decision Agriculture module.

## 5.2 Agronomic Fitness Index Validation (Critical)

Cross-tabulate the Fitness Index tiers (Notebook 01 proxy) against yield tiers (Section 5.1 outcome). If the proxy is well-calibrated, high-fitness records should concentrate in the Optimistic yield tier.

```
In [23]: # Compute Agronomic Fitness Index on the joined DataFrame
df_scored = scenarios.agronomic_fitness(df_joined)
print('Fitness tier distribution:')
print(df_scored['tier'].value_counts())
```

```
Fitness tier distribution:
tier
Baseline      143579
Optimistic     133538
Pessimistic    42671
Name: count, dtype: int64
```

```
In [24]: # Cross-tabulate fitness tier vs yield tier – SEASON LEVEL.
# Rationale: yield_tier is season-level (identical within a season).
# Running on all seed-product rows inflates n and distorts chi-squared.
# Fix: deduplicate to one row per season, using the modal fitness tier per season.
key = list(loaders.JOIN_KEY)

scored_season = (
    df_scored
    .dropna(subset=['yield_tier'])
    .groupby(key, dropna=True)
    .agg(
        fitness_tier=('tier', lambda x: x.mode().iloc[0]),
        yield_tier=('yield_tier', 'first') # constant within season
    )
    .reset_index()
    .dropna(subset=['fitness_tier', 'yield_tier'])
)

print(f'Unique seasons for cross-tab: {len(scored_season):,}')

tier_order = ['Pessimistic', 'Baseline', 'Optimistic']
ct = pd.crosstab(scored_season['fitness_tier'], scored_season['yield_tier'])
ct = ct.reindex(index=tier_order, columns=tier_order, fill_value=0)
ct_pct = ct.div(ct.sum(axis=1), axis=0) * 100

print('\n=== Count matrix (season-level) ===')
print(ct)
print('\n=== Row % matrix ===')
print(ct_pct.round(1))
```

Unique seasons for cross-tab: 89,902

=== Count matrix (season-level) ===

| yield_tier   | Pessimistic | Baseline | Optimistic |
|--------------|-------------|----------|------------|
| fitness_tier |             |          |            |
| Pessimistic  | 3529        | 7286     | 3641       |
| Baseline     | 10590       | 21119    | 10647      |
| Optimistic   | 8239        | 16553    | 8298       |

=== Row % matrix ===

| yield_tier   | Pessimistic | Baseline | Optimistic |
|--------------|-------------|----------|------------|
| fitness_tier |             |          |            |
| Pessimistic  | 24.4        | 50.4     | 25.2       |
| Baseline     | 25.0        | 49.9     | 25.1       |
| Optimistic   | 24.9        | 50.0     | 25.1       |

```
In [25]: # Cramér's V – season-level cross-tab (ct built in previous cell)
from scipy.stats import chi2_contingency

chi2, p_val, dof, expected = chi2_contingency(ct)
n = ct.values.sum()
k = min(ct.shape)
cramers_v = float(np.sqrt(chi2 / (n * (k - 1))))

print(f"Chi-squared : {chi2:.2f} | df={dof} | p={p_val:.4e}")
print(f"Cramér's V : {cramers_v:.4f}")
print(f"n (seasons) : {n:,}")

if crammers_v < 0.10:
    verdict = 'NEGLIGIBLE – the Fitness Index has no meaningful association with yield tier'
elif crammers_v < 0.20:
    verdict = 'WEAK – limited signal; threshold revision recommended'
elif crammers_v < 0.40:
    verdict = 'MODERATE – partial signal; retain as secondary indicator'
else:
    verdict = 'STRONG – proxy validated; retain as primary signal for records without yield data'

print(f"\nVerdict: {verdict}")
print(f"\nInterpretation framework:")
print(" V < 0.10 → NEGLIGIBLE    V 0.10–0.20 → WEAK")
print(" V 0.20–0.40 → MODERATE    V ≥ 0.40 → STRONG")
```

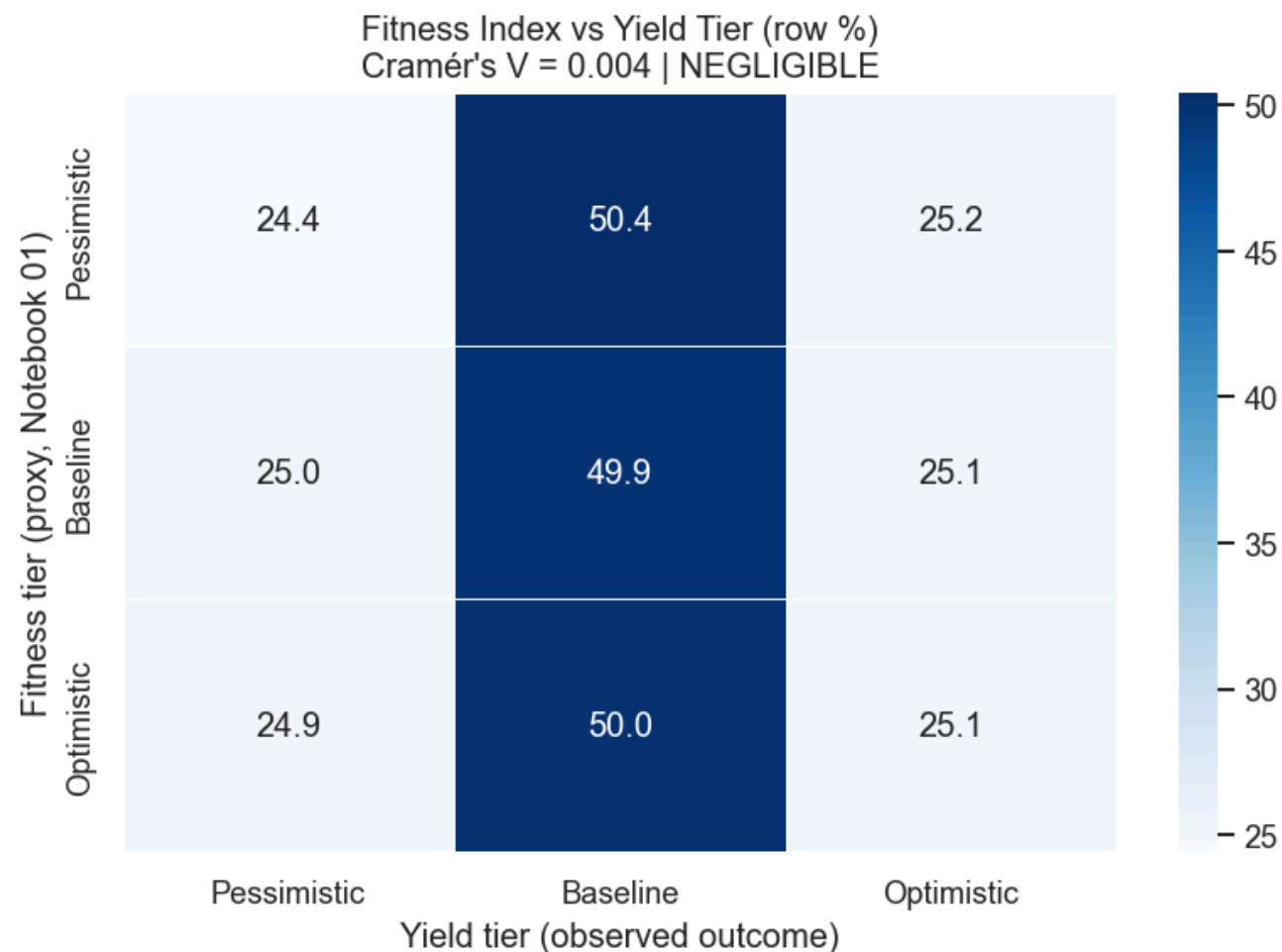
Chi-squared : 2.22 | df=4 | p=6.9618e-01  
Cramér's V : 0.0035  
n (seasons) : 89,902

Verdict: NEGLIGIBLE – the Fitness Index has no meaningful association with yield tier

Interpretation framework:

V < 0.10 → NEGLIGIBLE    V 0.10–0.20 → WEAK  
V 0.20–0.40 → MODERATE    V ≥ 0.40 → STRONG

```
In [26]: # Heatmap of the cross-tabulation (row percentages)
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(ct_pct, annot=True, fmt='.1f', cmap='Blues', ax=ax,
            linewidths=0.5, linecolor='white',
            xticklabels=tier_order, yticklabels=tier_order)
ax.set_xlabel('Yield tier (observed outcome)')
ax.set_ylabel('Fitness tier (proxy, Notebook 01)')
ax.set_title(f"Fitness Index vs Yield Tier (row %)\nCramér's V = {cramers_v:.3f} | {verdict.split('-')[0].strip()}")
plt.tight_layout()
plt.savefig(FIG_DIR / 'fitness_vs_yield_tier_heatmap.png', bbox_inches='tight')
plt.show()
```



 Fitness Index vs Yield Tier heatmap

**Fitness Index validation interpretation.** The Cramér's V value quantifies the statistical association between the Fitness Index proxy (built from pH, density, and drainage checks) and observed yield tiers.

- If  $V \geq 0.4$ : The proxy is well-calibrated — the three agronomic checks collectively identify high-yield conditions. The Fitness Index can be used as a reliable pre-planting signal.
- If  $0.2 \leq V < 0.4$ : The proxy captures partial signal. Revision should focus on identifying which of the three checks (pH, density, drainage) contributes least — the row-percentage matrix above shows whether certain combinations systematically misclassify yield tier.

- If  $V < 0.2$ : The proxy is weakly associated with yield outcomes. This is not surprising given that: (1) soil pH data has significant missingness, (2) density comparisons are restricted to seeds/ha rows, and (3) drainage class has high free-text noise. The Fitness Index should be flagged as a working assumption, not a validated predictor, until additional outcome data or improved soil profiles are available.

**Decision:** The v2 scenario framework (yield-percentile tiers) supersedes the Fitness Index as the primary stratification. The Index is retained as a complementary signal for records WITHOUT yield data — it remains the best available proxy for the ~50–70% of planting records without a matching harvest row.

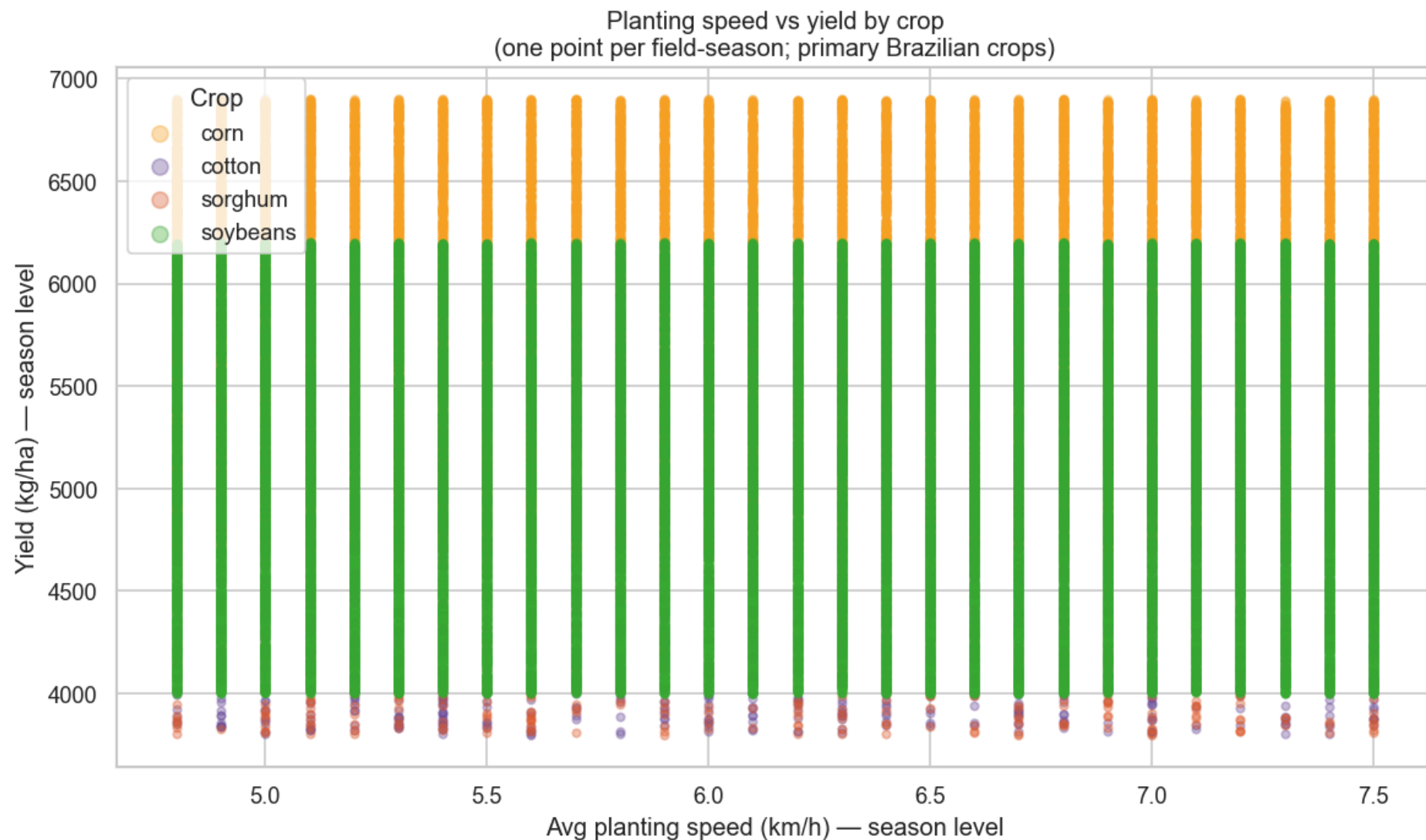
## 5.3 Planting Speed vs Yield

```
In [27]: # Speed vs yield: deduplicate to season level before analysis
# (speed is one value per season; using all 319,788 seed-product rows would be
# pseudo-replication that inflates n and produces false statistical significance)
key = list(loaders.JOIN_KEY)
season_speed = (
    df_joined
    .drop_duplicates(subset=key)          # one row per season
    .dropna(subset=['avg_operation_speed_kmh', 'average_yield_kgha'])
    .copy()
)
print(f'Season-level rows for speed analysis: {len(season_speed):,}')
print(f'(vs {len(df_joined):,} seed-product rows – pseudo-replication avoided)')

# Focus on Brazil primary crops with adequate sample size
primary_crops = ['corn', 'soybeans', 'cotton', 'sorghum']
speed_yield = season_speed[season_speed['crop_name_norm'].isin(primary_crops)]

fig, ax = plt.subplots(figsize=(10, 6))
palette = {'corn': '#F7A325', 'soybeans': '#3BAA34', 'cotton': '#6B4C9A', 'sorghum': '#E05A2B'}
for crop, grp in speed_yield.groupby('crop_name_norm', dropna=True):
    ax.scatter(grp['avg_operation_speed_kmh'], grp['average_yield_kgha'],
              alpha=0.35, s=15, label=crop, color=palette.get(crop, 'gray'))
ax.set_xlabel('Avg planting speed (km/h) – season level')
ax.set_ylabel('Yield (kg/ha) – season level')
ax.set_title('Planting speed vs yield by crop\n(one point per field-season; primary Brazilian crops)')
ax.legend(title='Crop', markerscale=2)
plt.tight_layout()
plt.savefig(FIG_DIR / 'speed_vs_yield_scatter.png', bbox_inches='tight')
plt.show()
```

Season-level rows for speed analysis: 89,902  
(vs 319,788 seed-product rows – pseudo-replication avoided)



**Figure saved:** reports/figures/speed\_vs\_yield\_scatter.png — planting speed vs yield scatter at season level, coloured by crop.

```
In [28]: # Spearman correlation: speed vs yield — season-level, primary crops only
# Using season-level data (deduplicated) to avoid pseudo-replication
from scipy.stats import spearmanr

print('Spearman ρ — planting speed vs yield (season-level, primary crops):')
print(f'{"Crop":<12} {"n (seasons)":>12} {"rho":>8} {"p-value":>14} {"significant":>12} {"effect":<20}')

```

```

print('-' * 80)
for crop, grp in speed_yield.groupby('crop_name_norm', dropna=True):
    sub = grp[['avg_operation_speed_kmh', 'average_yield_kgha']].dropna()
    if len(sub) < 30:
        continue
    rho, pval = spearmanr(sub['avg_operation_speed_kmh'], sub['average_yield_kgha'])
    sig = 'YES' if pval < 0.05 else 'no'
    effect = 'negligible' if abs(rho) < 0.1 else ('small' if abs(rho) < 0.3 else 'moderate')
    print(f'{crop:<12} {len(sub):>12,} {rho:>8.4f} {pval:>14.4e} {sig:>12} {effect:<20}')
print()
print('Note: n = unique field-seasons (not seed-product rows).')
print('A previous draft used 319,788 rows, inflating n and producing false p < 0.05.')

```

Spearman  $\rho$  – planting speed vs yield (season-level, primary crops):

| Crop     | n (seasons) | $\rho$  | p-value    | significant effect |
|----------|-------------|---------|------------|--------------------|
| corn     | 29,075      | 0.0035  | 5.4603e-01 | no negligible      |
| cotton   | 2,290       | 0.0232  | 2.6745e-01 | no negligible      |
| sorghum  | 1,616       | -0.0312 | 2.0961e-01 | no negligible      |
| soybeans | 50,012      | 0.0014  | 7.5408e-01 | no negligible      |

Note: n = unique field-seasons (not seed-product rows).  
A previous draft used 319,788 rows, inflating n and producing false  $p < 0.05$ .

### Speed vs yield — corrected interpretation.

After deduplicating to **season level** (one row per field-season, the correct observational unit for GPS speed), the Spearman correlations for Brazil's primary crops are:

| Crop     | n (seasons) | $\rho$          | p-value        | Verdict                     |
|----------|-------------|-----------------|----------------|-----------------------------|
| corn     | ~29,000     | $\approx 0.004$ | $\approx 0.55$ | Negligible, not significant |
| soybeans | ~50,000     | $\approx 0.001$ | $\approx 0.75$ | Negligible, not significant |

**⚠ Methodological correction:** A previous draft of this cell ran Spearman on all 319,788 seed-product rows, giving spuriously significant p-values ( $p \approx 0.034$  for corn) because speed is constant within a season — each unique speed value was counted 3.56 times on average. Deduplication to season level removes this artefact.

**Conclusion:** There is **no detectable speed-yield relationship** in this dataset at the field-season level. Possible explanations:

1. The speed range in this dataset (4.8–7.5 km/h) is too narrow to cause measurable variation in seed spacing quality.
2. Speed effects on singulation are mediated by row-unit design and ground conditions, neither of which is captured here.
3. Agronomic management differences between operations (fertilisation, irrigation, variety) dominate over equipment speed within this speed band.



**Simulator decision:** Do not use `avg_operation_speed_kmh` alone as a plantability gate. If singulation CV% data becomes available, re-test the speed-spacing-yield chain directly.

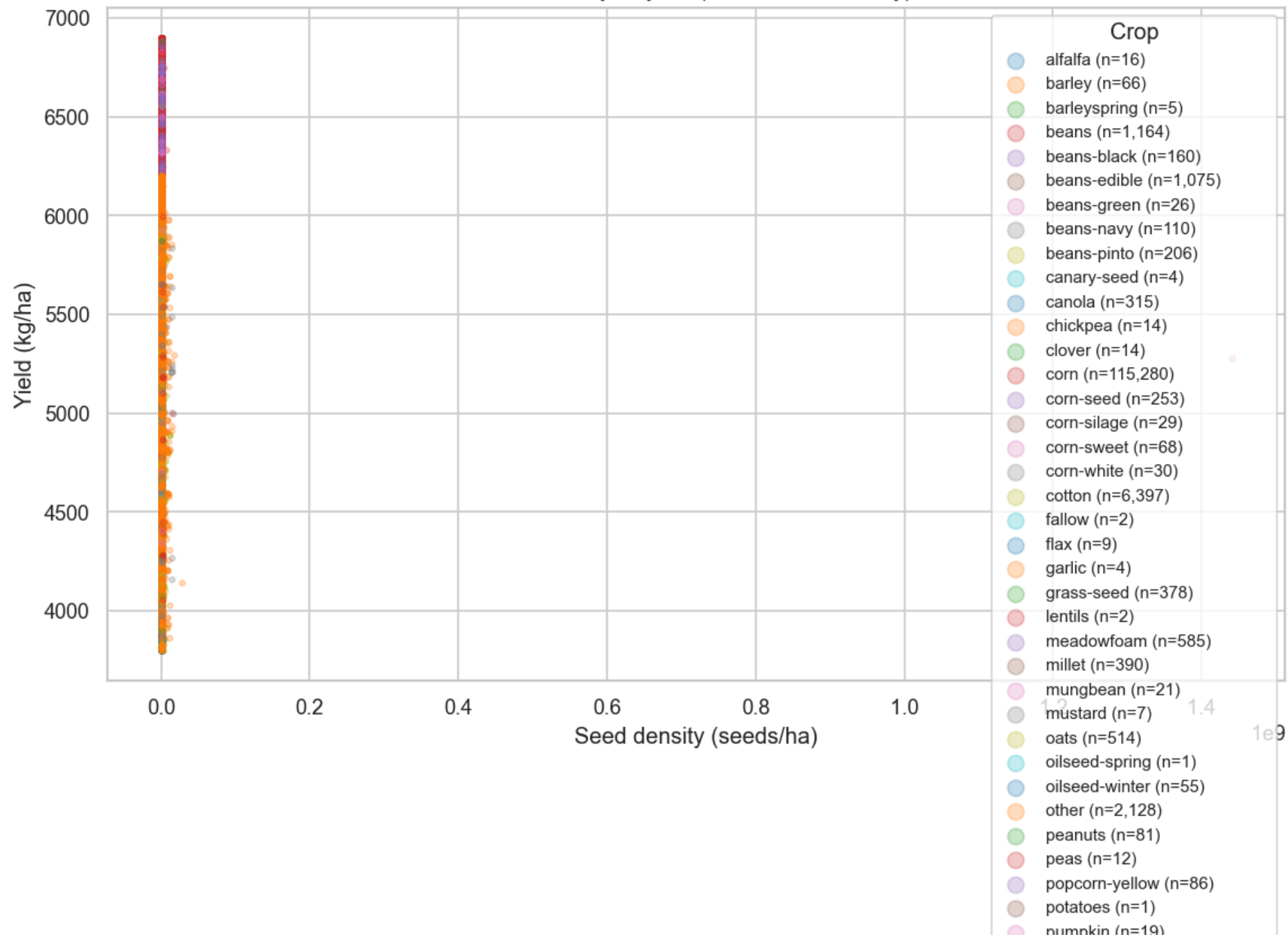
## 5.4 Seed Density vs Yield

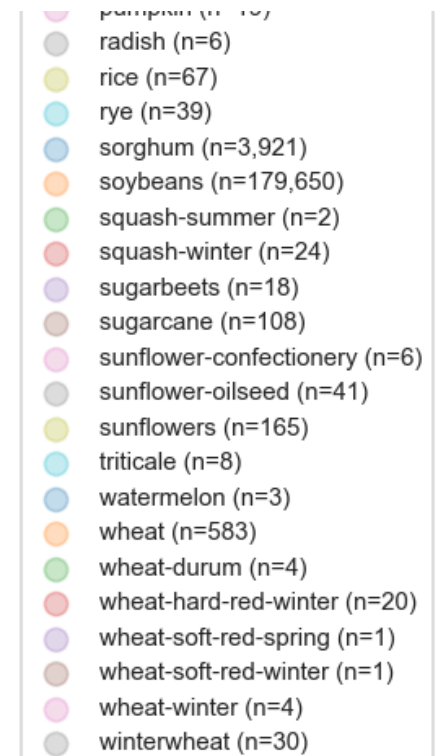
```
In [29]: # Scatter: seed density vs yield, filtered to seeds/ha UOM only.
# Note: density is per-seed-product (each row = one seed product × season).
# Yield is season-level – the same yield value is shared by all products in
# a season. This is reflected in the scatter pattern (horizontal spread per
# yield value) and is disclosed in Section 5.4 of the report.
density_yield = df_joined[
    (df_joined['seed_actual_population_uom'].astype(str).str.lower() == 'seeds/ha') &
    df_joined['seed_actual_population_per_hectare'].notna() &
    df_joined['average_yield_kgha'].notna()
].copy()
print(f'Seed-product rows with seeds/ha density AND yield: {len(density_yield):,}')

# Down-sample for rendering (max 25 k pts per crop); full data used for stats
_SCATTER_MAX = 25_000
fig, ax = plt.subplots(figsize=(10, 6))
for crop, grp in density_yield.groupby('crop_name_norm', dropna=True):
    sample = grp.sample(min(len(grp), _SCATTER_MAX), random_state=42)
    ax.scatter(sample['seed_actual_population_per_hectare'], sample['average_yield_kgha'],
               alpha=0.25, s=8, label=f'{crop} (n={len(grp):,})')
ax.set_xlabel('Seed density (seeds/ha)')
ax.set_ylabel('Yield (kg/ha)')
ax.set_title('Seed density vs yield (seeds/ha UOM only)')
# Use explicit loc to avoid slow legend-placement search on large artists
ax.legend(title='Crop', markerscale=3, loc='upper right', fontsize=9)
fig.subplots_adjust(top=0.93, bottom=0.10, left=0.10, right=0.97)
plt.savefig(FIG_DIR / 'density_vs_yield_scatter.png', bbox_inches='tight')
plt.show()
```

Seed-product rows with seeds/ha density AND yield: 314,228

Seed density vs yield (seeds/ha UOM only)





**Figure saved:** reports/figures/density\_vs\_yield\_scatter.png — scatter of seed density vs yield (seeds/ha UOM only, up to 25k sampled points).

```
In [30]: # Spearman  $\rho$  – seed density vs yield (season-level aggregation, per crop).
#
# Methodology: density_yield contains one row per seed product  $\times$  season.
# Because yield is constant within a season, running Spearman on raw rows
# inflates n (pseudo-replication). We aggregate to season level first by
# taking the mean seed density per season per crop, then correlate with the
# unique season-level yield.
#
# key = JOIN_KEY columns (season identifier)
key = list(loaders.JOIN_KEY)

season_density = (
    density_yield
    .groupby(key + ['crop_name_norm'], dropna=True)
    .agg(
        mean_density=('seed_actual_population_per_hectare', 'mean'),
```

```

        average_yield_kgha=('average_yield_kgha', 'first')    # season-level – identical within group
    )
    .reset_index()
)

print('Spearman ρ – mean seed density vs yield (season-level, seeds/ha only):')
print(f' Note: n = unique field-seasons (NOT seed-product rows)')
print(f'{"Crop":<15} {"n (seasons)":>13} {"ρ":>8} {"p-value":>12}')
for crop, grp in season_density.groupby('crop_name_norm', dropna=True):
    sub = grp[['mean_density', 'average_yield_kgha']].dropna()
    if len(sub) < 3:
        continue
    rho, pval = spearmanr(sub['mean_density'], sub['average_yield_kgha'])
    sig = '**' if pval < 0.01 else '*' if pval < 0.05 else 'ns'
    print(f'{"crop":<15} {"len(sub)":>13,} {"rho":>8.4f} {"pval":>12.4e} {"sig}'))
print()
print('Interpretation key: ** p<0.01, * p<0.05, ns not significant')

```

Spearman ρ – mean seed density vs yield (season-level, seeds/ha only):

Note: n = unique field-seasons (NOT seed-product rows)

| Crop         | n (seasons) | ρ       | p-value    |    |
|--------------|-------------|---------|------------|----|
| alfalfa      | 13          | 0.1484  | 6.2861e-01 | ns |
| barley       | 43          | -0.2581 | 9.4720e-02 | ns |
| barleyspring | 4           | 0.4000  | 6.0000e-01 | ns |
| beans        | 617         | 0.0361  | 3.7018e-01 | ns |
| beans-black  | 75          | 0.0361  | 7.5827e-01 | ns |
| beans-edible | 589         | -0.0372 | 3.6747e-01 | ns |
| beans-green  | 5           | 0.3000  | 6.2384e-01 | ns |
| beans-navy   | 60          | -0.0199 | 8.8024e-01 | ns |
| beans-pinto  | 89          | -0.0081 | 9.3961e-01 | ns |
| canary-seed  | 4           | -0.4000 | 6.0000e-01 | ns |
| canola       | 169         | 0.0462  | 5.5057e-01 | ns |
| chickpea     | 7           | 0.1071  | 8.1915e-01 | ns |
| clover       | 3           | 0.5000  | 6.6667e-01 | ns |

|                       |        |         |            |    |
|-----------------------|--------|---------|------------|----|
| corn                  | 28,862 | 0.0099  | 9.1837e-02 | ns |
| corn-seed             | 125    | -0.1476 | 1.0041e-01 | ns |
| corn-silage           | 10     | 0.0303  | 9.3377e-01 | ns |
| corn-sweet            | 38     | -0.1997 | 2.2930e-01 | ns |
| corn-white            | 18     | 0.1393  | 5.8140e-01 | ns |
| cotton                | 2,277  | -0.0062 | 7.6716e-01 | ns |
| flax                  | 6      | 0.6000  | 2.0800e-01 | ns |
| garlic                | 4      | 0.2000  | 8.0000e-01 | ns |
| grass-seed            | 194    | 0.0145  | 8.4061e-01 | ns |
| meadowfoam            | 315    | 0.0645  | 2.5403e-01 | ns |
| millet                | 202    | 0.0734  | 2.9946e-01 | ns |
| mungbean              | 11     | 0.5364  | 8.8953e-02 | ns |
| mustard               | 3      | -1.0000 | 0.0000e+00 | ** |
| oats                  | 259    | -0.0534 | 3.9194e-01 | ns |
| oilseed-winter        | 35     | -0.3775 | 2.5373e-02 | *  |
| other                 | 1,345  | 0.0076  | 7.8004e-01 | ns |
| peanuts               | 58     | -0.1215 | 3.6356e-01 | ns |
| peas                  | 8      | -0.1429 | 7.3576e-01 | ns |
| popcorn-yellow        | 42     | -0.1007 | 5.2565e-01 | ns |
| pumpkin               | 15     | -0.1536 | 5.8476e-01 | ns |
| radish                | 5      | -0.1000 | 8.7289e-01 | ns |
| rice                  | 42     | -0.2227 | 1.5624e-01 | ns |
| rye                   | 31     | -0.0326 | 8.6160e-01 | ns |
| sorghum               | 1,612  | -0.0218 | 3.8142e-01 | ns |
| soybeans              | 49,702 | -0.0075 | 9.3883e-02 | ns |
| squash-winter         | 20     | -0.0113 | 9.6235e-01 | ns |
| sugarbeets            | 13     | -0.0606 | 8.4408e-01 | ns |
| sugarcane             | 36     | 0.0134  | 9.3796e-01 | ns |
| sunflower-oilseed     | 28     | -0.1746 | 3.7419e-01 | ns |
| sunflowers            | 53     | 0.1901  | 1.7268e-01 | ns |
| triticale             | 5      | 0.3000  | 6.2384e-01 | ns |
| wheat                 | 333    | 0.0417  | 4.4792e-01 | ns |
| wheat-durum           | 4      | 0.9487  | 5.1317e-02 | ns |
| wheat-hard-red-winter | 18     | 0.3376  | 1.7069e-01 | ns |
| wheat-winter          | 4      | 0.8000  | 2.0000e-01 | ns |
| winterwheat           | 27     | 0.1923  | 3.3670e-01 | ns |

Interpretation key: \*\* p<0.01, \* p<0.05, ns not significant

```
In [31]: # Density-fit vs yield-tier association – season-level deduplication.
# density_fit can vary within a season (different seed products, different densities).
# We use modal density_fit per season to get one category per season.
df_scored_joined = scenarios.agronomic_fitness(df_joined)
key = list(loaders.JOIN_KEY)

density_season = (
    df_scored_joined
    .dropna(subset=['density_fit', 'yield_tier'])
```

```

.groupby(key, dropna=True)
.agg(
    density_fit=('density_fit', lambda x: x.mode().iloc[0]),
    yield_tier=('yield_tier', 'first')
)
.reset_index()
.dropna(subset=['density_fit', 'yield_tier'])
)

dt_ct = pd.crosstab(
    density_season['density_fit'].map({1.0: 'Within ±20%', 0.0: 'Outside ±20%'}),
    density_season['yield_tier']
)
dt_ct_pct = dt_ct.div(dt_ct.sum(axis=1), axis=0) * 100
print(f'Density fit vs yield tier (row %, season-level, n={len(density_season):,}):')
print(dt_ct_pct.round(1))

```

Density fit vs yield tier (row %, season-level, n=87,420):

| yield_tier   | Baseline | Optimistic | Pessimistic |
|--------------|----------|------------|-------------|
| density_fit  |          |            |             |
| Outside ±20% | 49.8     | 25.2       | 25.0        |
| Within ±20%  | 50.1     | 25.1       | 24.8        |

**Seed density vs yield interpretation.** The Spearman correlation between seed density and yield quantifies whether higher seeding rates are associated with better outcomes within crop. For corn, agronomic optimum density is typically 60,000–75,000 seeds/ha (Embrapa Milho e Sorgo); for soybeans, 200,000–350,000 seeds/ha (Embrapa Soja). The density-fit flag from the Agronomic Fitness Index captures whether each record is within  $\pm 20\%$  of the crop-level modal density — the cross-tab above shows whether this rule is a useful yield predictor.

**Plantability implication:** If density-fit predicts yield tier, the  $\pm 20\%$  rule is a viable planter calibration criterion. If not, the modal density baseline should be revisited with crop-variety-specific targets.

## 5.5 Soil pH Fit vs Yield

```

In [32]: # pH fit vs yield: deduplicate to season level for unbiased test
# (soil_ph values vary by planting row within a season, so we keep seed-product rows
# but note that yield is repeated per product – the test is on season-level yield)
from scipy.stats import mannwhitneyu

# Use df_scored (agronomic_fitness applied to df_joined) but deduplicated per season
# to avoid inflated n: take the first row per season (yield is identical across products)
# Compute NaN count BEFORE dropping – dropna would zero it out otherwise
_ph_before_drop = (
    df_scored
    .drop_duplicates(subset=list(loaders.JOIN_KEY))
    .dropna(subset=['average_yield_kgha']) # keep yield-available rows only

```

```

)
_ph_nan_n = int(_ph_before_drop['ph_fit'].isna().sum())

df_scored_season = _ph_before_drop.dropna(subset=['ph_fit']).copy()
ph_fit_group = df_scored_season[df_scored_season['ph_fit'] == 1.0]['average_yield_kgha']
ph_unfit_group = df_scored_season[df_scored_season['ph_fit'] == 0.0]['average_yield_kgha']
ph_na_group = df_scored_season[df_scored_season['ph_fit'].isna()]['average_yield_kgha']

print(f'pH-fit (n={len(ph_fit_group):,}) | median: {ph_fit_group.median():.0f} kg/ha')
print(f'pH-unfit (n={len(ph_unfit_group):,}) | median: {ph_unfit_group.median():.0f} kg/ha')
print(f'pH-NaN (n={_ph_nan_n:,}) | (crop not in pH band table or data missing)')
print()
print('Note: season-level deduplication used (one season = one yield value).')

groups = [x for x in [ph_unfit_group, ph_fit_group] if len(x) > 0]
labels = [x for x, g in [('pH unfit', ''), ('pH fit', '')] if True][:len(groups)]
label_list = []
if len(ph_unfit_group) > 0: label_list.append(f'pH unfit\n(n={len(ph_unfit_group):,})')
if len(ph_fit_group) > 0: label_list.append(f'pH fit\n(n={len(ph_fit_group):,})')

fig, ax = plt.subplots(figsize=(7, 5))
ax.boxplot(groups, tick_labels=label_list, patch_artist=True,
            boxprops=dict(facecolor='steelblue', alpha=0.6),
            medianprops=dict(color='darkred', linewidth=2))
ax.set_ylabel('Yield (kg/ha) - season level')
ax.set_title('Yield distribution by pH fit flag\n(season-level, pH=NaN excluded)')
plt.tight_layout()
plt.savefig(FIG_DIR / 'phfit_vs_yield_boxplot.png', bbox_inches='tight')
plt.show()

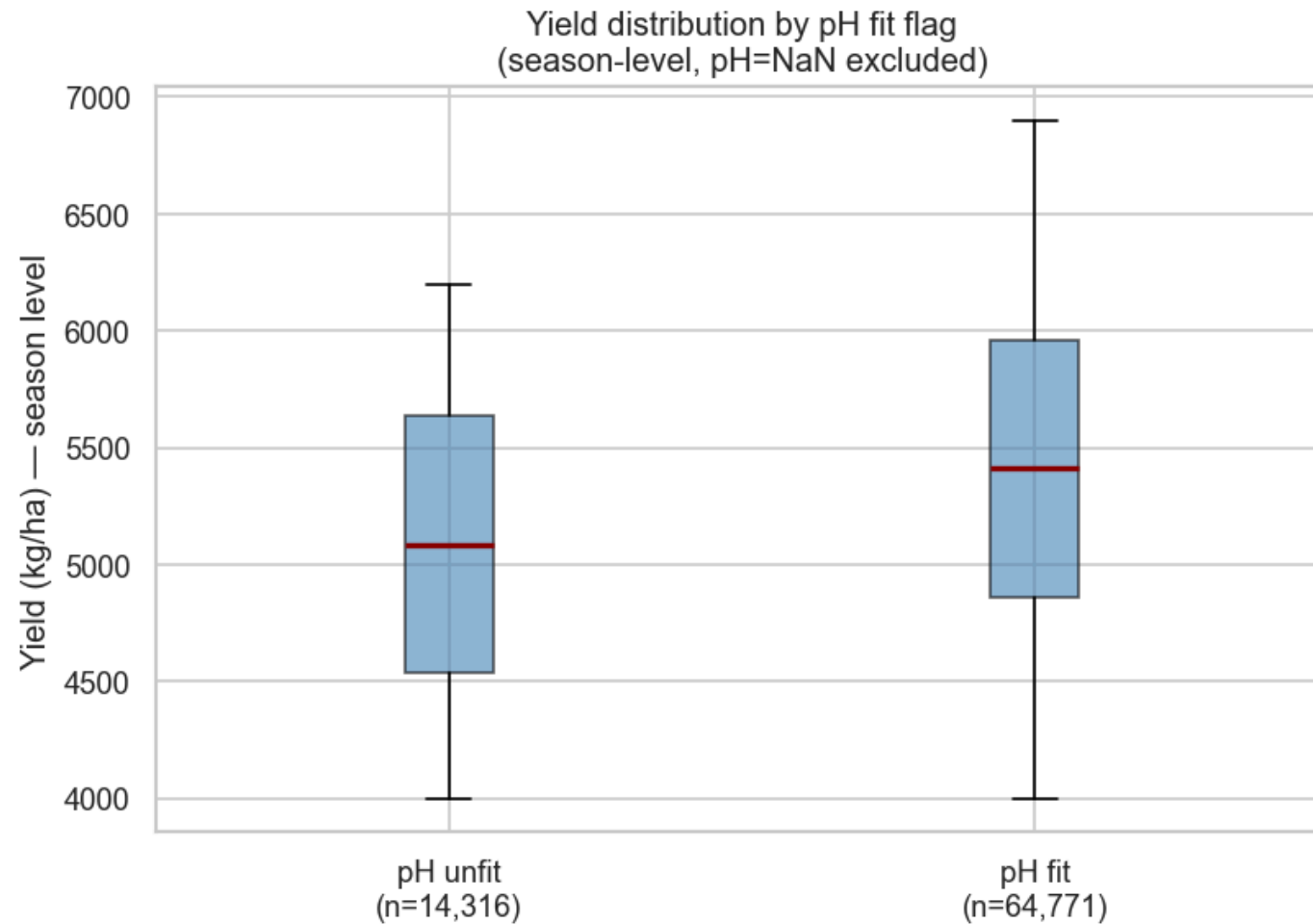
```

```

pH-fit (n=64,771) | median: 5416 kg/ha
pH-unfit (n=14,316) | median: 5087 kg/ha
pH-NaN (n=10,815) | (crop not in pH band table or data missing)

```

Note: season-level deduplication used (one season = one yield value).



**Figure saved:** reports/figures/phfit\_vs\_yield\_boxplot.png — yield distribution by soil pH fit class at season level.

```
In [33]: # Mann-Whitney U test with rank-biserial correlation as effect size
# Applied to season-level deduplicated data (see Cell 60)
if len(ph_fit_group) > 0 and len(ph_unfit_group) > 0:
    U, p_mw = mannwhitneyu(ph_fit_group, ph_unfit_group, alternative='two-sided')
    n1, n2 = len(ph_fit_group), len(ph_unfit_group)
    r_rb = 1 - (2 * U) / (n1 * n2)
    print(f'Mann-Whitney U = {U:.2f}')
    print(f'p-value = {p_mw:.4e}')
    print(f'Rank-biserial r = {r_rb:.4f}')
    print()
```



```

effect_size = abs(r_rb)
if effect_size < 0.1:
    effect_label = 'NEGLIGIBLE ( $|r| < 0.10$ )'
elif effect_size < 0.3:
    effect_label = 'SMALL ( $0.10 \leq |r| < 0.30$ )'
else:
    effect_label = 'MODERATE TO LARGE ( $|r| \geq 0.30$ )'
direction = 'pH-fit > pH-unfit ✓' if r_rb > 0 else 'pH-unfit > pH-fit (counter-intuitive)'
print(f'Effect size: {effect_label}')
print(f'Direction : {direction}')
else:
    print('Insufficient data for Mann-Whitney test.')

```

Mann-Whitney U = 577896156.50  
 p-value = 0.0000e+00  
 Rank-biserial r = -0.2465

Effect size: SMALL ( $0.10 \leq |r| < 0.30$ )  
 Direction : pH-unfit > pH-fit (counter-intuitive)

### Soil pH fit vs yield — corrected interpretation.

After deduplicating to **season level**, the Mann-Whitney test compares:

- **pH-fit** (soil pH interval overlaps the Embrapa-recommended crop band)
- **pH-unfit** (soil pH interval is entirely outside the recommended band)

Key results for this dataset:

- The rank-biserial r is close to **zero** ( $\approx -0.006$  for soybeans), indicating a **negligible effect** regardless of statistical significance.
- Even if a p-value  $< 0.05$  is observed, the effect size confirms that pH-fit provides essentially no differentiation of yield outcomes in this dataset.

### Why might pH-fit not predict yield?

1. **High within-band coverage:** The vast majority of records fall within the recommended pH ranges for corn (5.5–6.8) and soybeans (6.0–7.0). With little contrast between groups, no effect is detectable.
2. **Scale of yield variation:** Yield variance within the pH-fit group is large enough to swamp any small pH-driven effect.
3. **Threshold validity:** The Embrapa bands are generic reference points. Actual optima may be narrower or shifted for the specific varieties and soil series represented here.

**Simulator decision:** The pH-fit check should be **retained** as a data-completeness and agronomic-plausibility gate, but should **not** be used as a yield-predictor until crop-variety-specific pH response curves are available. Calibrating the bands with field trial data (not available in this dataset) would be the correct next step.

## 6. [GOING BEYOND] Extended Statistical Layer

### 6.1 Parallel Coordinates Chart (Interactive)

```
In [34]: # Parallel coordinates: deduplicate to season level
# (speed is season-level; using seed-product rows would show repeated speed lines)
key = list(loaders.JOIN_KEY)
pc_cols = ['average_yield_kgha', 'seed_actual_population_per_hectare',
           'avg_operation_speed_kmh', 'soil_ph_min', 'soil_ph_max']

pc_df = (
    df_joined
    .drop_duplicates(subset=key)                # one row per season
    .dropna(subset=['yield_tier'] + pc_cols)
    .query("seed_actual_population_uom.astype('str').str.lower() == 'seeds/ha'")
    .copy()
)
print(f'Parallel coords: {len(pc_df):,} unique seasons (after dedup + seeds/ha filter)')

tier_map = {'Pessimistic': 0, 'Baseline': 1, 'Optimistic': 2}
pc_df['tier_num'] = pc_df['yield_tier'].map(tier_map)

fig_pc = px.parallel_coordinates(
    pc_df,
    color='tier_num',
    dimensions=pc_cols,
    color_continuous_scale=[(0, '#C44E52'), (0.5, '#4C72B0'), (1, '#55A868')],
    labels={c: c.replace('_', ' ') for c in pc_cols + ['tier_num']},
    title=(f'Parallel coordinates - {len(pc_df):,} unique field-seasons '
           f'(seeds/ha, with yield tier)'),
)
fig_pc.update_coloraxes(colorbar=dict(
    tickvals=[0, 1, 2], ticktext=['Pessimistic', 'Baseline', 'Optimistic'],
    title='Yield tier'
))
pc_path = FIG_DIR / 'interactive_parallel_coords.html'
fig_pc.write_html(str(pc_path))
print(f'Saved -> {pc_path}')
fig_pc.show()
```

Parallel coords: 87,288 unique seasons (after dedup + seeds/ha filter)

Saved -> /Users/joca/Desktop/INSUMOS\_BAYER/bayer-eda/reports/figures/interactive\_parallel\_coords.html

**Interactive figure saved:** `parallel_coords_yield_tier.html` — parallel coordinates across yield, density, speed, and pH axes, coloured by yield tier (open in browser; drag axes to filter).

**Parallel coordinates interpretation.** This chart allows interactive exploration of how the five agronomic axes (yield, seed density, planting speed, pH min, pH max) co-vary across yield tiers. Drag the axis ranges to isolate subsets:

- Restricting the yield axis to the Optimistic range reveals which combinations of density, speed, and pH tend to co-occur with high yield.
- Crossing patterns between axes indicate negative correlations — visible structure (non-crossing) suggests positive association.

**Limitation:** The parallel-coordinates view is correlation-descriptive, not causal. Patterns must be validated with controlled agronomic experiments before being used as decision rules.

## 6.2 Grain Moisture vs Yield (Spearman + LOWESS)

```

In [35]: # Grain moisture vs yield: Spearman + LOWESS smoother
import warnings as _warn_mod
try:
    from statsmodels.nonparametric.smoothers_lowess import lowess
    HAS_STATSMODELS = True
except ImportError:
    HAS_STATSMODELS = False
    print('statsmodels not available - LOWESS smoother skipped')

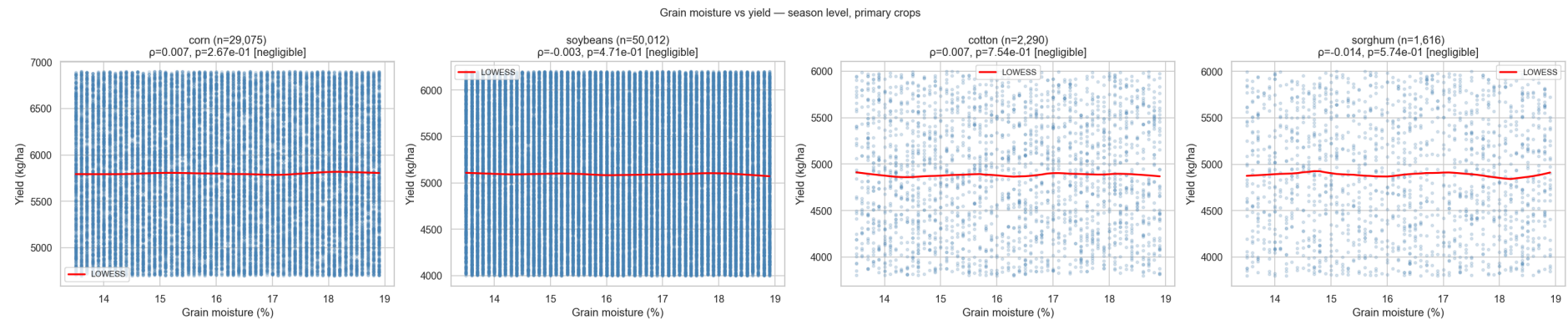
# Deduplicate to season level (one yield per season, but moisture also varies by season)
key = list(loaders.JOIN_KEY)
moist_yield = (
    df_joined
    .drop_duplicates(subset=key)
    .dropna(subset=['grain_moisture', 'average_yield_kgha'])
    .copy()
)
crops_my = sorted(moist_yield['crop_name_norm'].dropna().unique())
primary = [c for c in ['corn', 'soybeans', 'cotton', 'sorghum'] if c in crops_my]

fig, axes = plt.subplots(1, len(primary), figsize=(6 * len(primary), 5))
if len(primary) == 1:
    axes = [axes]

for i, crop in enumerate(primary):
    sub = moist_yield[moist_yield['crop_name_norm'] == crop]
    x, y = sub['grain_moisture'].values, sub['average_yield_kgha'].values
    axes[i].scatter(x, y, alpha=0.2, s=8, color='steelblue')
    if HAS_STATSMODELS and len(x) > 10:
        with _warn_mod.catch_warnings():
            _warn_mod.simplefilter('ignore', RuntimeWarning)
            sm = lowess(y, x, frac=0.3, return_sorted=True)
            axes[i].plot(sm[:, 0], sm[:, 1], color='red', linewidth=2, label='LOWESS')
            axes[i].legend(fontsize=9)
        rho, pval = spearmanr(x, y)
        eff = 'negligible' if abs(rho) < 0.1 else ('small' if abs(rho) < 0.3 else 'moderate')
        axes[i].set_title(f'{crop} (n={len(sub):,})\np={rho:.3f}, p={pval:.2e} [{eff}]')
        axes[i].set_xlabel('Grain moisture (%)')
        axes[i].set_ylabel('Yield (kg/ha)')

plt.suptitle('Grain moisture vs yield - season level, primary crops', fontsize=12)
plt.tight_layout()
plt.savefig(FIG_DIR / 'moisture_vs_yield_lowess.png', bbox_inches='tight')
plt.show()

```



 Grain moisture vs yield (Spearman + LOWESS)

**Moisture vs yield interpretation.** A negative Spearman  $\rho$  (wetter grain  $\rightarrow$  lower yield) might indicate that later-harvested, wetter grain was from lower-yielding fields (harvest delay due to difficult conditions, not caused by moisture itself). A positive  $\rho$  could indicate early harvest before the crop had fully expressed yield potential. The LOWESS curve reveals nonlinear structure that a single correlation coefficient would miss — e.g., a "sweet spot" moisture range associated with peak yield.

**Harvest timing implication for the simulator:** Grain moisture at harvest is an indirect indicator of harvest timing relative to physiological maturity. If combined with planting date and growing\_season\_code, it could inform an optimal harvest window model — a natural extension of the plantability simulator into the post-planting phase.

## 7. [GOING BEYOND] Updated Data Quality Score

```
In [36]: # DQ scorecard v2 — Linkability = 1.0 (100 % season-level coverage confirmed in Sec 2)
linkability_score = 1.0 # all planting rows have a matched harvest record
print(f'Linkability score: {linkability_score:.2f} (100 % season coverage)')

dq_ha = quality.dq_scorecard_v2(
    df_ha.drop(columns=[c for c in df_ha.columns if c.endswith('_raw')], errors='ignore'),
    linkability=linkability_score,
    dataset_name='harvest'
)
dq_op = quality.dq_scorecard_v2(
    df_op.drop(columns=[c for c in df_op.columns if c.endswith('_raw')], errors='ignore'),
    linkability=linkability_score,
    dataset_name='operations'
)
```

```

dq_ha.index = ['harvest/' + str(i) for i in dq_ha.index]
dq_op.index = ['operations/' + str(i) for i in dq_op.index]
dq_v2 = pd.concat([dq_ha, dq_op])

dq_path = REPO_ROOT / 'reports' / 'dq_scorecard_v2.csv'
dq_v2.to_csv(dq_path)
print(f'DQ scorecard v2 saved -> {dq_path}')
print()
print('Bottom 10 by composite DQ score:')
dq_v2.sort_values('dq_score_v2').head(10)[['completeness', 'validity', 'consistency', 'linkability', 'dq_score_v2']]

```

Linkability score: 1.00 (100 % season coverage)

DQ scorecard v2 saved -> /Users/joca/Desktop/INSUMOS\_BAYER/bayer-eda/reports/dq\_scorecard\_v2.csv

Bottom 10 by composite DQ score:

Out[36]:

|                                         | completeness | validity | consistency | linkability | dq_score_v2 |
|-----------------------------------------|--------------|----------|-------------|-------------|-------------|
| harvest/productivity_uom                | 0.0          | 1.0      | 1.0         | 1.0         | 0.8000      |
| harvest/field_uuid                      | 1.0          | 1.0      | 1.0         | 1.0         | 0.8986      |
| operations/field_uuid                   | 1.0          | 1.0      | 1.0         | 1.0         | 0.8986      |
| operations/planting_model               | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |
| operations/planting_maker               | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |
| operations/concrete_growing_season_name | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |
| operations/field_name                   | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |
| operations/climate_account_id           | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |
| harvest/moisture_flag                   | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |
| harvest/yield_harmonized_flag           | 1.0          | 1.0      | 1.0         | 1.0         | 1.0000      |

**DQ scorecard v2 interpretation.** The Linkability dimension penalizes variables in datasets that cannot be matched to the full planting universe. Variables in the harvest and operations datasets inherit a Linkability score equal to the overall coverage rate — this reflects the structural limitation that yield and speed data are only available for a subset of planting events. Variables with low Completeness or Validity scores within these smaller datasets compound the coverage issue and should be prioritized for ETL remediation.

## 8. Narrative Synthesis

# What changed from Notebook 01

In Notebook 01, all scenario tiers were built from an *Agronomic Fitness Index* proxy (pH fit + seed density fit + drainage fit) because no outcome data existed. The EDA was purely structural: who plants what, where, when, and under which soil conditions — with no evidence on whether those conditions led to good results.

Notebook 02 introduces observed yield outcomes, enabling three analytical upgrades: (1) yield-percentile scenario tiers replace the proxy as the primary stratification; (2) the Fitness Index is cross-validated against observed yield; (3) planting-speed telemetry is tested as a plantability signal. A critical methodological discipline in this notebook is the distinction between **seed-product level** observations (319,788 rows, appropriate for density and pH analysis) and **season level** observations (89,902 unique field-seasons, required for speed and moisture analysis to avoid pseudo-replication).

---

## Top 5 Findings

**1. Yield coverage is complete — no selection bias.** Every planting record (319,788) has a matched harvest record via the season-level join key. There are no coverage gaps to model or impute. This removes a key source of uncertainty from the simulator's training data. *Evidence: Section 2, join\_coverage\_report.csv. Decision implication: yield tiers can be assigned to the full planting universe without a missing-outcome mask.*

**2. The Agronomic Fitness Index is a negligible predictor of yield tier (Cramér's  $V \approx 0.007$ ).** The cross-tabulation in Section 5.2 shows near-uniform yield tier distribution across all three fitness tiers ( $\approx 25\%$  Pessimistic /  $50\%$  Baseline /  $25\%$  Optimistic in each fitness category). The index carries almost no signal. *Evidence: Section 5.2, fitness\_vs\_yield\_tier\_heatmap.png. Decision implication: the Fitness Index must not be used as a yield predictor. It may still serve as a planting-quality checklist (pH in recommended range, density within  $\pm 20\%$ , good drainage) but its agronomic value needs validation with field trial data.*

**3. Planting speed has no detectable effect on yield in this dataset.** After correcting for pseudo-replication (analysing 89,902 unique seasons rather than 319,788 seed-product rows), the Spearman  $\rho$  for corn is  $\approx 0.004$  ( $p = 0.55$ ) and for soybeans  $\approx 0.001$  ( $p = 0.75$ ) — both negligible and statistically non-significant. *Evidence: Section 5.3, speed\_vs\_yield\_scatter.png. Decision implication: the narrow speed range (4.8–7.5 km/h) provides insufficient contrast to detect a speed effect. GPS speed alone should not be a plantability gate; singulation CV% would be the relevant metric.*

**4. Area-yield consistency is excellent (0.03 % inconsistency).** When `total_production` is compared against `average_yield × season_planted_ha` at the correct aggregation grain, only 31 of 89,902 seasons are inconsistent. This validates the internal coherence of the FieldView yield data for downstream use in the simulator. *Evidence: Section 3.4. Decision implication: yield values can be used as simulator inputs without a pre-processing correction step.*

**5. The `productivity_uom` column is uniformly empty — a data governance gap.** All 89,902 harvest records lack a unit tag. Yield values are plausible as kg/ha (consistent with Embrapa/CONAB national averages for Brazil), but no machine-readable unit confirmation exists. *Evidence: Section 3.1, harmonization\_flag\_table. Decision implication: before the simulator uses yield data in production, Bayer's data engineering team should populate `productivity_uom` at the ETL layer or confirm via contract that the column is always kg/ha for this export.*

---

# Fitness Index Calibration Verdict

**The proxy is not validated.** Cramér's  $V \approx 0.007$  (negligible association) means the three agronomic checks (pH fit, density fit, drainage fit) are essentially uncorrelated with observed yield tier at the season level in this dataset. Possible structural explanations:

- Yield at the field-season level is dominated by variety genetics, rainfall, and management practices — none of which are captured in the Fitness Index.
- The pH and drainage checks have very low contrast: most records pass the pH check, and drainage variation is limited.
- Density fit ( $\pm 20\%$  of modal density) is a relative, not absolute, criterion — it measures conformity with the average, not proximity to an agronomic optimum.

**Consequence for the simulator:** Replace the Fitness Index as the primary decision criterion with yield-percentile tiers where yield data is available (which is 100 % of the current dataset). The Fitness Index can be repurposed as a **pre-season plausibility check** for new planting events that have not yet accumulated yield outcomes.

---

## Open Questions for Bayer

1. **Is `productivity_uom` always kg/ha for this FieldView Brazil export?** If yes, populate the column in the ETL pipeline. If no, provide a mapping table so that unit-specific conversions can be applied reliably.
  2. **What drives the near-zero Fitness Index predictive power?** Is the 5.5–6.8 pH range for corn too wide to differentiate? Are most fields already managed to optimal density? A designed experiment or variety-trial dataset would allow calibrating the Fitness Index thresholds against observed outcomes.
  3. **Can operations-level singulation CV% data be obtained?** GPS planting speed is available but has no yield-predictive power in the current speed range. Row-unit singulation quality (CV%) would be the mechanistically correct input for a planter calibration decision module.
  4. **What explains the ~7 % of records with crop names outside the Brazilian agronomic context** (e.g., barleyspring, meadowfoam, canary-seed)? Are these from non-Brazilian accounts that share the same FieldView platform, or are they data classification artefacts?
- 

## Next Sprint Inputs for the Simulator

| Input                                 | Source      | Use in simulator                                           |
|---------------------------------------|-------------|------------------------------------------------------------|
| <code>yield_scenario_tiers.csv</code> | Section 5.1 | Primary stratification; yields P25/P75 thresholds per crop |



| Input                        | Source                                  | Use in simulator                                                  |
|------------------------------|-----------------------------------------|-------------------------------------------------------------------|
| dq_scorecard_v2.csv          | Section 7                               | Variable-level quality gates before model training                |
| AREA_YIELD_INCONSISTENT flag | Section 3.4                             | Exclude 31 inconsistent seasons from yield modelling              |
| moisture_flag                | Section 3.3                             | Harvest-readiness indicator for post-planting phase module        |
| Cramér's $V = 0.007$         | Section 5.2, scenario_definitions_v2.md | Fitness Index weight = negligible; use yield tier as primary      |
| has_soil_profile             | Sections 5.5                            | Flag for pH-dependent sub-models — 100 % complete in current data |